

Bottom-Up Parsing

- | | |
|--|--|
| 5.1 Overview of Bottom-Up Parsing | 5.5 Yacc: An LALR(1) Parser Generator |
| 5.2 Finite Automata of LR(0) Items and LR(0) Parsing | 5.6 Generation of a TINY Parser Using Yacc |
| 5.3 SLR(1) Parsing | 5.7 Error Recovery in Bottom-Up Parsers |
| 5.4 General LR(1) and LALR(1) Parsing | |
-

In the previous chapter we covered the basic top-down parsing algorithms of recursive-descent and predictive parsing. In this chapter, we will describe the major bottom-up parsing techniques and their associated constructions. As with top-down parsing, we restrict ourselves primarily to the study of those parsing algorithms that *make use of at most one symbol of lookahead*, with a few comments on how to extend the algorithms:

In terminology similar to that of LL(1) parsers, the most general bottom-up algorithm is called **LR(1) parsing** (the L indicates that the input is processed from left to right, the R indicates that a rightmost derivation is produced, and the number 1 indicates that one symbol of lookahead is used). A consequence of the power of bottom-up parsing is the fact that it is also meaningful to talk about **LR(0) parsing**, where *no* lookahead is consulted in making parsing decisions. (This is possible because a lookahead token can be examined *after* it appears on the parsing stack, and if this happens it does not count as lookahead.) An improvement on LR(0) parsing that does make some use of lookahead is called **SLR(1) parsing** (for *simple* LR(1) parsing). A method that is slightly more powerful than SLR(1) parsing but less complex than general LR(1) parsing is called **LALR(1) parsing** (for *lookahead* LR(1) parsing).

We will treat the constructions necessary for each of these parsing methods in the following sections. This will include the construction of the DFAs of LR(0) and LR(1) items; descriptions of the SLR(1), LR(1), and LALR(1) parsing algorithms; and the construction of the associated parsing tables. We will also describe the use of Yacc, an LALR(1) parser generator, and will use Yacc to generate a parser for the

TINY language that constructs the same syntax trees as the recursive-descent parser that we developed in the previous chapter.

Bottom-up parsing algorithms are in general more powerful than top-down methods. (For example, left recursion is not a problem in bottom-up parsing.) Not unexpectedly, the constructions involved in these algorithms are also more complex. Thus, we will need to proceed carefully in their description, and we will need to use very simple examples of grammars to introduce them. We will give two such examples early in the chapter, which we will use throughout the rest of the chapter. We will also continue with the use of some of the recurrent examples of the previous chapter (integer arithmetic expressions, if-statements, etc.). We will not carry out by hand *any* of the bottom-up parsing algorithms for the full TINY language, however, since this would be tremendously complex. Indeed, all of the important bottom-up methods are really too complex for hand coding, but are well suited for parser generators like Yacc. Nevertheless, it is important to understand the operation of the methods so that the behavior of a parser generator can be properly analyzed by the compiler writer. The designer of a programming language can also benefit from this information, since a parser generator can identify potential problems with a proposed language syntax in BNF.

Previously covered topics that will be needed in order to understand the operation of bottom-up parsing algorithms include the properties of finite automata and the subset construction of a DFA from an NFA (Chapter 2, Sections 2.3 and 2.4) and general properties of context-free grammars, derivations, and parse trees (Chapter 3, Sections 3.2 and 3.3). Follow sets (Chapter 4, Section 4.3) will also be needed occasionally. We begin this chapter with a general overview of bottom-up parsing.

5.1 OVERVIEW OF BOTTOM-UP PARSING

A bottom-up parser uses an explicit stack to perform a parse, similar to a nonrecursive top-down parser. The parsing stack will contain both tokens and nonterminals, and also some extra state information that we will discuss later. The stack is empty at the beginning of a bottom-up parse and will contain the start symbol at the end of a successful parse. A schematic for bottom-up parsing is, therefore,

\$	<i>InputString</i>	\$
...	...	
...	...	
\$ <i>StartSymbol</i>		\$ accept

where the parsing stack is on the left, the input is in the center, and the actions of the parser are on the right (in this case, “accept” is the only action indicated).

A bottom-up parser has two possible actions (besides “accept”):

1. **Shift** a terminal from the front of the input to the top of the stack.
2. **Reduce** a string α at the top of the stack to a nonterminal A , given the BNF choice $A \rightarrow \alpha$.

A bottom-up parser is thus sometimes called a **shift-reduce** parser.¹ Shift actions are indicated by writing the word *shift*. Reduce actions are indicated by writing the word *reduce* and giving the BNF choice used in the reduction.² One further feature of bottom-up parsers is that, for technical reasons to be discussed later, grammars are always **augmented** with a new start symbol. This means that if S is the start symbol, a new start symbol S' is added to the grammar, with a single unit production to the previous start symbol:

$$S' \rightarrow S$$

We proceed immediately to two examples, and then discuss some of the properties of bottom-up parsing displayed by these examples.

Example 5.1

Consider the following augmented grammar for balanced parentheses:

$$\begin{aligned} S' &\rightarrow S \\ S &\rightarrow (S) S \mid \varepsilon \end{aligned}$$

A bottom-up parse of the string $()$ using this grammar is given in Table 5.1.

Table 5.1

Parsing actions of a bottom-up parser for the grammar of Example 5.1	Parsing stack	Input	Action
1	\$	$()$ \$	shift
2	$\$ ($	$)$ \$	reduce $S \rightarrow \varepsilon$
3	$\$ (S$	$)$ \$	shift
4	$\$ (S)$	\$	reduce $S \rightarrow \varepsilon$
5	$\$ (S) S$	\$	reduce $S \rightarrow (S) S$
6	$\$ S$	\$	reduce $S' \rightarrow S$
7	$\$ S'$	\$	accept

§

Example 5.2

Consider the following augmented grammar for rudimentary arithmetic expressions (no parentheses and one operation):

$$\begin{aligned} E' &\rightarrow E \\ E &\rightarrow E + n \mid n \end{aligned}$$

A bottom-up parse of the string $n + n$ using this grammar is given in Table 5.2.

1. Top-down parsers could be called generate-match parsers for the same reason, but this is not customary.

2. In the case of a reduction, we could simply write the BNF choice by itself, as we did for a *generate* action in top-down parsing, but it is customary to add the *reduce*.

Table 5.2

Parsing actions of a bottom-up parser for the grammar of Example 5.2		Parsing stack	Input	Action
	1	\$	$n + n$ \$	shift
	2	\$ n	$+ n$ \$	reduce $E \rightarrow n$
	3	\$ E	$+ n$ \$	shift
	4	\$ $E +$	n \$	shift
	5	\$ $E + n$	\$	reduce $E \rightarrow E + n$
	6	\$ E	\$	reduce $E' \rightarrow E$
	7	\$ E'	\$	accept

§

Bottom-up parsers have less difficulty than top-down parsers with lookahead. Indeed, a bottom-up parser can shift input symbols onto the stack until it determines what action to perform (assuming that an action can be determined that does not require the symbols to be shifted back to the input). However, a bottom-up parser may need to look deeper into the stack than just the top in order to determine what action to perform. For example, in Table 5.1, line 5 has S on the top of the stack, and the parser performs a reduction by the production $S \rightarrow (S) S$, while line 6 also has S on the top of the stack, but the parser performs a reduction by $S' \rightarrow S$. To be able to know that $S \rightarrow (S) S$ is a valid reduction at step 5, we must know that the stack actually does contain the string $(S) S$ at that point. Thus, bottom-up parsing requires arbitrary “stack lookahead.” This is not nearly as serious as input lookahead, since the parser itself builds the stack and can arrange for the appropriate information to be available. The mechanism that will do this is a deterministic finite automaton of “items” that will be described in the next section.

Of course, keeping track of the stack contents alone is not enough to be able to uniquely determine the next step in a shift-reduce parse—the next token in the input may also need to be consulted as a lookahead. For example, in line 3 of Table 5.2, E is on the stack and a shift occurs, while in line 6, E is again on the stack but a reduction by $E' \rightarrow E$ occurs. The difference is that in line 3 the next token in the input is $+$, while in line 6 the next input token is $\$$. Thus, any algorithm that performs that parse must make use of the next input token (the lookahead) to determine the appropriate action. Different shift-reduce parsing methods use the lookahead in different ways, and this results in parsers of varying power and complexity. Before we look at the individual algorithms, we make some general observations on how the intermediate stages in a bottom-up parse may be characterized.

First, we note again that a shift-reduce parser traces out a rightmost derivation of the input string, but the steps of the derivation occur in reverse order. In Table 5.1 there are four reductions, corresponding in reverse to the four steps of the rightmost derivation:

$$S' \Rightarrow S \Rightarrow (S) S \Rightarrow (S) \Rightarrow ()$$

In Table 5.2, the corresponding derivation is

$$E' \Rightarrow E \Rightarrow E + n \Rightarrow n + n$$

Each of the intermediate strings of terminals and nonterminals in such a derivation is called a **right sentential form**. Each such sentential form is split between the parsing stack and the input during a shift-reduce parse. For example, the right sentential form

$E + \mathbf{a}$, which occurs in the third step of the previous derivation, occurs in steps 3, 4, and 5 of Table 5.2. If we indicate where the top of the stack is at each moment by the symbol $\parallel$ (that is, where the split occurs between the stack and the input), then step 3 of Table 5.2 is given by $E \parallel + \mathbf{a}$ and step 4 is given by $E + \parallel \mathbf{a}$. In each case, the sequence of symbols on the parsing stack is called a **viable prefix** of the right sentential form. Thus, E , $E +$, and $E + \mathbf{a}$ are all viable prefixes of the right sentential form $E + \mathbf{a}$, while the right sentential form $\mathbf{a} + \mathbf{a}$ has ε and $\mathbf{a}$ as its viable prefixes (steps 1 and 2 of Table 5.2). Note that $\mathbf{a} +$ is not a viable prefix of $\mathbf{a} + \mathbf{a}$.

A shift-reduce parser will shift terminals from the input to the stack until it is possible to perform a reduction to obtain the next right sentential form. This will occur when the string of symbols on the top of the stack matches the right-hand side of the production that is used in the next reduction. This string, together with the position in the right sentential form where it occurs, and the production used to reduce it, is called the **handle** of the right sentential form.³ For example, in the right sentential form $\mathbf{a} + \mathbf{a}$, the handle is the string consisting of the leftmost single token $\mathbf{a}$, together with the production $E \rightarrow \mathbf{a}$ that is used to reduce it to yield the new right sentential form $E + \mathbf{a}$. The handle of this new sentential form is the entire string $E + \mathbf{a}$ (a viable prefix), together with the production $E \rightarrow E + \mathbf{a}$. Sometimes, by abuse of notation, we refer to the string by itself as the handle.

Determining the next handle in a parse is the main task of a shift-reduce parser. Note that the string of a handle always forms a complete right-hand side for its production (the production used in the next reduction) and that the rightmost position of the handle string will correspond to the top of the stack when the reduction is to take place. Thus, it seems plausible that a shift-reduce parser will want to determine its actions based on positions in right-hand sides of productions. When these positions reach the right-hand end of a production, then this production is a candidate for a reduction, and it is possible that the handle is at the top of the stack. To be the handle, however, it is not enough for the string at the top of the stack to match the right-hand side of a production. Indeed, if an ε -production is available for reduction, as in Example 5.1, then its right-hand side (the empty string) is always at the top of the stack. Reductions occur only when the resulting string is indeed a right sentential form. For example, in step 3 of Table 5.1 a reduction by $S \rightarrow \varepsilon$ could be performed, but the resulting string $(S S)$ is not a right sentential form, and thus ε is not the handle at this position in the sentential form (S) .

5.2 FINITE AUTOMATA OF LR(0) ITEMS AND LR(0) PARSING

5.2.1 LR(0) Items

An **LR(0) item** (or just **item** for short) of a context-free grammar is a production choice with a distinguished position in its right-hand side. We will indicate this distinguished position by a period (which, of course, becomes a metasymbol, not to be confused with

3. If the grammar is ambiguous, so that more than one derivation may exist, then there may be more than one handle in a right sentential form. If the grammar is unambiguous, then handles are unique.

an actual token). Thus, if $A \rightarrow \alpha$ is a production choice, and if β and γ are any two strings of symbols (including the empty string ϵ) such that $\beta\gamma = \alpha$, then $A \rightarrow \beta \cdot \gamma$ is an LR(0) item. These are called LR(0) items because they contain no explicit reference to lookahead.

Example 5.3

Consider the grammar of Example 5.1:

$$\begin{aligned} S' &\rightarrow S \\ S &\rightarrow (S) S \mid \epsilon \end{aligned}$$

This grammar has three production choices and eight items:

$$\begin{aligned} S' &\rightarrow \cdot S \\ S' &\rightarrow S \cdot \\ S &\rightarrow \cdot (S) S \\ S &\rightarrow (\cdot S) S \\ S &\rightarrow (S \cdot) S \\ S &\rightarrow (S) \cdot S \\ S &\rightarrow (S) S \cdot \\ S &\rightarrow \cdot \end{aligned}$$

§

Example 5.4

The grammar of Example 5.2 has the following eight items:

$$\begin{aligned} E' &\rightarrow \cdot E \\ E' &\rightarrow E \cdot \\ E &\rightarrow \cdot E + n \\ E &\rightarrow E \cdot + n \\ E &\rightarrow E + \cdot n \\ E &\rightarrow E + n \cdot \\ E &\rightarrow \cdot n \\ E &\rightarrow n \cdot \end{aligned}$$

§

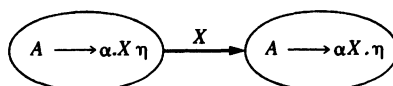
The idea behind the concept of an item is that an item records an intermediate step in the recognition of the right-hand side of a particular grammar rule choice. In particular, the item $A \rightarrow \beta \cdot \gamma$ constructed from the grammar rule choice $A \rightarrow \alpha$ (with $\alpha = \beta\gamma$) means that β has already been seen and that it may be possible to derive the next input tokens from γ . In terms of the parsing stack, this means that β must appear at the top of the stack. An item $A \rightarrow \cdot \alpha$ means that we may be about to recognize an A by using the grammar rule choice $A \rightarrow \alpha$ (we call such items **initial items**). An item $A \rightarrow \alpha \cdot$ means that α now resides on the top of the parsing stack and may be the handle, if $A \rightarrow \alpha$ is to be used for the next reduction (we call such items **complete items**).

5.2.2 Finite Automata of Items

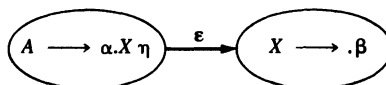
The LR(0) items can be used as the states of a finite automaton that maintains information about the parsing stack and the progress of a shift-reduce parse. This will start

out as a nondeterministic finite automaton. From this NFA of LR(0) items we can construct the DFA of sets of LR(0) items using the subset construction of Chapter 2. As we will see, it is also easy to construct the DFA of sets of LR(0) items directly.

What are the transitions of the NFA of LR(0) items? Consider the item $A \rightarrow \alpha \cdot \gamma$, and suppose γ begins with the symbol X , which may be either a token or a nonterminal, so that the item can be written as $A \rightarrow \alpha \cdot X\eta$. Then there is a transition on the symbol X from the state represented by this item to the state represented by the item $A \rightarrow \alpha X \cdot \eta$. In graphical form we write this as



If X is a token, then this transition corresponds to a shift of X from the input to the top of the stack during a parse. On the other hand, if X is a nonterminal, then the interpretation of this transition is more complex, since X will never appear as an input symbol. In fact, such a transition will still correspond to the pushing of X onto the stack during a parse, but this can only occur during a reduction by a production $X \rightarrow \beta$. Now, since such a reduction must be preceded by the recognition of a β , and the state given by the initial item $X \rightarrow \cdot \beta$ represents the beginning of this process (the dot indicating that we are about to recognize a β), then for every item $A \rightarrow \alpha \cdot X\eta$ we must add an ϵ -transition



for every production choice $X \rightarrow \beta$ of X , indicating that X can be produced by recognizing any of the right-hand sides of its production choices.

These two cases represent the only transitions in the NFA of LR(0) items. It remains to discuss the choice of initial state and accepting states of the NFA. The start state of the NFA should correspond to the initial state of the parser: the stack is empty, and we are about to recognize an S , where S is the start symbol of the grammar. Thus, any initial item $S \rightarrow \cdot \alpha$ constructed from a production choice for S could serve as a start state. Unfortunately, there may be many such production choices for S . How can we tell which one to use? In fact, we can't. The solution is to **augment** the grammar by a single production $S' \rightarrow S$, where S' is a new nonterminal. S' then becomes the start state of the **augmented grammar**, and the initial item $S' \rightarrow \cdot S$ becomes the start state of the NFA. This is the reason we have augmented the grammars of the previous examples.

What states should become accepting states in this NFA? Here we must remember that the purpose of the NFA is to keep track of the state of a parse, not to recognize strings outright, as the automata in Chapter 2 are designed to do. Thus, the parser itself will decide when to accept, and the NFA need not contain that information, so the NFA will in fact have no accepting states at all. (The NFA *will* have some information on acceptance but not in the form of an accepting state. We will discuss this when we describe the parsing algorithms that use the automaton.)

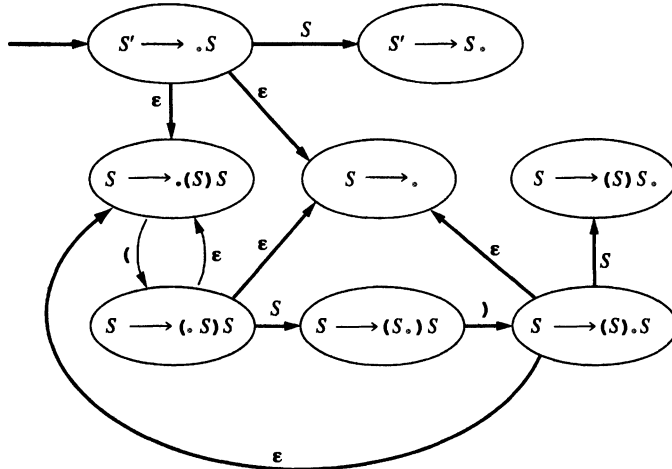
This completes the description of the NFA of LR(0) items. We now turn to the simple grammars of the two previous examples and construct their NFAs of LR(0) items.

Example 5.5

In Example 5.3 we listed the eight LR(0) items of the grammar of Example 5.1. The NFA, therefore, has eight states; it is shown in Figure 5.1. Note that every item in the figure with a dot before the nonterminal S has an ϵ -transition to every initial item of S .

Figure 5.1

The NFA of LR(0) items for the grammar of Example 5.5



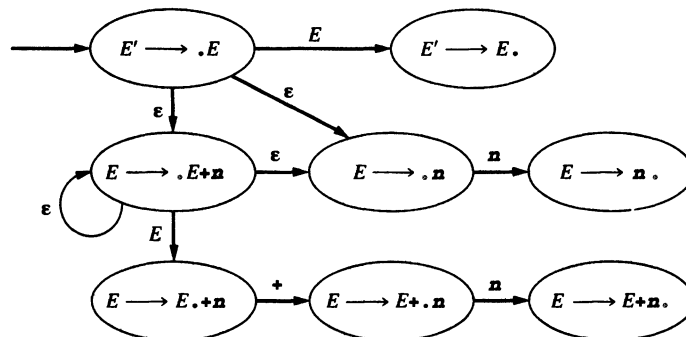
§

Example 5.6

In Example 5.4 we listed the LR(0) items associated with the grammar of Example 5.2. The NFA of items appears in Figure 5.2. Note that the initial item $E \rightarrow \cdot E + n$ has an ϵ -transition to itself. (This situation will occur in all grammars with immediate left recursion.)

Figure 5.2

The NFA of LR(0) items for the grammar of Example 5.6



§

To complete the description of the use of items to keep track of the parsing state, we must construct the DFA of sets of items corresponding to the NFA of items according to the subset construction of Chapter 2. We will then be able to state the LR(0) parsing algorithm. We perform the subset construction for the two examples of NFAs that we have just given.

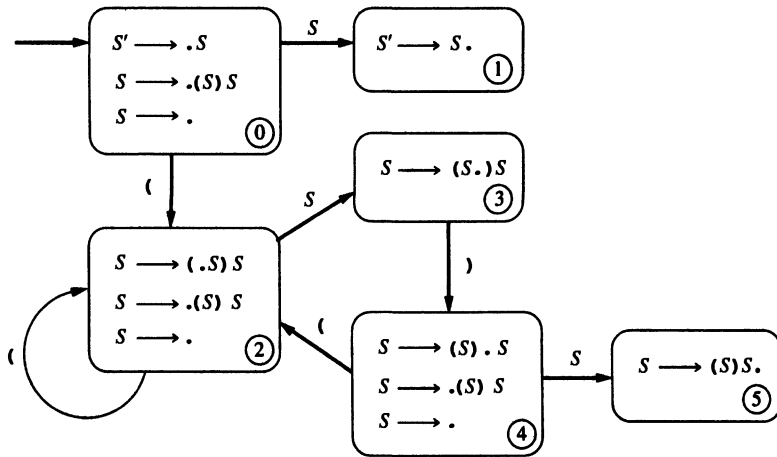
Example 5.7

Consider the NFA of Figure 5.1. The start state of the associated DFA is the ϵ -closure of the set consisting of the item $S' \rightarrow \cdot S$, and this is the set of three items $\{ S' \rightarrow \cdot S, S \rightarrow \cdot (S) S, S \rightarrow (\cdot S) S \}$.

$S \rightarrow \cdot (S) S, S \rightarrow \cdot)$. Since there is a transition from $S' \rightarrow \cdot S$ to $S' \rightarrow S \cdot$ on S , there is a corresponding transition from the start state to the DFA state $\{ S' \rightarrow S \cdot \}$ (there being no ϵ -transitions from $S' \rightarrow S \cdot$ to any other item). There is also a transition on $($ from the start state to the DFA state $\{ S \rightarrow \cdot (S) S, S \rightarrow \cdot (S) S, S \rightarrow \cdot \}$ (the ϵ -closure of $\{ S \rightarrow \cdot (S) S \}$). The DFA state $\{ S \rightarrow \cdot (S) S, S \rightarrow \cdot (S) S, S \rightarrow \cdot \}$ has transitions to itself on $($ and to $\{ S \rightarrow (\cdot S) S \}$ on S . This state has a transition to the state $\{ S \rightarrow (S) \cdot S, S \rightarrow \cdot (S) S, S \rightarrow \cdot \}$ on $)$. Finally, this last state has transitions on $($ to the previously constructed state $\{ S \rightarrow (\cdot S) S, S \rightarrow \cdot (S) S, S \rightarrow \cdot \}$ and a transition on S to the state $\{ S \rightarrow (S) S \cdot \}$. The complete DFA is given in Figure 5.3, where we have numbered the states for reference (state 0 being traditionally the start state).

Figure 5.3

The DFA of sets of LR(0) items corresponding to the NFA of Figure 5.1

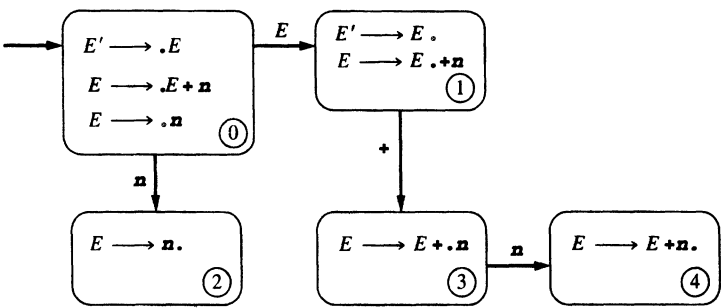


§

Example 5.8

Consider the NFA of Figure 5.2. The start state of the associated DFA consists of the set of three items $\{ E' \rightarrow \cdot E, E \rightarrow \cdot E + n, E \rightarrow \cdot n \}$. There is a transition from the item $E' \rightarrow \cdot E$ to the item $E' \rightarrow E \cdot$ on E , but there is also a transition on E from the item $E \rightarrow \cdot E + n$ to the item $E \rightarrow E \cdot + n$. Thus, there is a transition on E from the start state of the DFA to the closure of the set $\{ E' \rightarrow E \cdot, E \rightarrow E \cdot + n \}$. Since there are no ϵ -transitions from either of these items, this set is its own ϵ -closure and forms a complete DFA state. There is one other transition from the start state, corresponding to the transition on the symbol n from $E \rightarrow \cdot n$ to $E \rightarrow n \cdot$. Since there are no ϵ -transitions from the item $E \rightarrow n \cdot$, this item is its own ϵ -closure and forms the DFA state $\{ E \rightarrow n \cdot \}$. There are no transitions from this state, so the only transitions still to be computed are from the set $\{ E' \rightarrow E \cdot, E \rightarrow E \cdot + n \}$. There is only one transition from this set, corresponding to the transition from the item $E \rightarrow E \cdot + n$ to the item $E \rightarrow E + \cdot n$ on the symbol $+$. The item $E \rightarrow E + \cdot n$ also has no ϵ -transitions and so forms a singleton set in the DFA. Finally, there is a transition on n from the set $\{ E \rightarrow E + \cdot n \}$ to the set $\{ E \rightarrow E + n \cdot \}$. The entire DFA is given in Figure 5.4.

Figure 5.4
The DFA of sets of items
corresponding to the NFA
of Figure 5.2



§

In the construction of the DFA of sets of LR(0) items, a distinction is sometimes made between those items that are added to a state during the ϵ -closure step and those that originate the state as targets of non- ϵ -transitions. The former are called **closure items**, while the latter are called **kernel items**. In state 0 of Figure 5.4, $E' \rightarrow \cdot E$ is a kernel item (the only one), and $E \rightarrow \cdot E + n$ and $E \rightarrow \cdot n$ are closure items. In state 2 of Figure 5.3, $S \rightarrow (\cdot S) S$ is a kernel item, and $S \rightarrow \cdot (S) S$ and $S \rightarrow \cdot$ are closure items. By definition of the ϵ -transitions of the NFAs of items, all closure items are initial items.

The importance of the distinction between kernel and closure items is that, given the grammar, the kernel items uniquely determine the state and its transitions. Thus, only kernel items need to be specified to completely characterize the DFA of sets of items. Parser generators that construct the DFA may, therefore, only report the kernel items (this is true of Yacc, for instance).

A further simplification occurs if the DFA of sets of items is computed directly, rather than first computing the NFA of items and then applying the subset construction. Indeed, from a set of items it is easy to determine immediately what the ϵ -transitions are, and to which initial items they point. Therefore, parser generators like Yacc always compute the DFA directly from the grammar, and we will do this also for the remainder of the chapter.

5.2.3 The LR(0) Parsing Algorithm

We are now ready to state the LR(0) parsing algorithm. Since the algorithm depends on keeping track of the current state in the DFA of sets of items, we must modify the parsing stack to store not only symbols but also state numbers. We do this by pushing the new state number onto the parsing stack after each push of a symbol. In fact, the states themselves contain all the information about the symbols, so we could dispense with the symbols altogether and keep only state numbers on the parsing stack. We will, however, retain the symbols on the stack for convenience and clarity.

To begin a parse, we push the bottom marker \$ and the start state 0 onto the stack, so that at the beginning of the parse, the situation can be represented as

Parsing stack	Input
\$ 0	InputString \$

Suppose now that the next step is to shift a token x onto the stack and go to state 2 (this will in fact happen when the DFA is as in Figure 5.4 and x is the next token in the input). This is represented as follows:

Parsing stack	Input
\$ 0 x 2	Rest of InputString \$

The LR(0) parsing algorithm chooses an action based on the current DFA state, which is always the state that appears at the top of the stack.

The LR(0) parsing algorithm. Let s be the current state (at the top of the parsing stack). Then actions are defined as follows:

1. If state s contains any item of the form $A \rightarrow \alpha.X\beta$, where X is a terminal, then the action is to shift the current input token onto the stack. If this token is X , and state s contains item $A \rightarrow \alpha.X\beta$, then the new state to be pushed on the stack is the state containing the item $A \rightarrow \alpha.X.\beta$. If this token is not X for some item in state s of the form just described, an error is declared.
 2. If state s contains any complete item (an item of the form $A \rightarrow \gamma.$), then the action is to reduce by the rule $A \rightarrow \gamma$. A reduction by the rule $S' \rightarrow S$, where S is the start state, is equivalent to acceptance, provided the input is empty, and error if the input is not empty. In all other cases, the new state is computed as follows. Remove the string γ and all of its corresponding states from the parsing stack (the string γ must be at the top of the stack, according to the way the DFA is constructed). Correspondingly, back up in the DFA to the state from which the construction of γ began (this must be the state uncovered by the removal of γ). Again, by the construction of the DFA, this state must contain an item of the form $B \rightarrow \alpha.A\beta$. Push A onto the stack, and push (as the new state) the state containing the item $B \rightarrow \alpha.A.\beta$. (Note that this corresponds to following the transition on A in the DFA, which is indeed reasonable, since we are pushing A onto the stack.)
-

A grammar is said to be an **LR(0) grammar** if the above rules are unambiguous. This means that if a state contains a complete item $A \rightarrow \alpha.$, then it can contain no other items. Indeed, if such a state also contains a “shift” item $A \rightarrow \alpha.X\beta$ (X a terminal), then an ambiguity arises as to whether action (1) or action (2) is to be performed. This situation is called a **shift-reduce conflict**. Similarly, if such a state contains another complete item $B \rightarrow \beta.$, then an ambiguity arises as to which production to use for the reduction ($A \rightarrow \alpha$ or $B \rightarrow \beta$). This situation is called a **reduce-reduce conflict**. Thus, a grammar is LR(0) if and only if each state is a shift state (a state containing only “shift” items) or a reduce state containing a single complete item.

We note that neither of the two grammars we have been using as our examples are LR(0) grammars. In Figure 5.3 states 0, 2, and 4 all contain shift-reduce conflicts for the LR(0) parsing algorithm, while in the DFA of Figure 5.4, state 1 contains a shift-reduce conflict. This is not surprising, since almost all “real” grammars are not LR(0). We present, however, the following example of a grammar that is LR(0).

Example 5.9

Consider the grammar

$$A \rightarrow (A) \mid a$$

The augmented grammar has the DFA of sets of items shown in Figure 5.5, which is LR(0). To see how the LR(0) parsing algorithm works, consider the string $((a))$. A parse of this string according to the LR(0) parsing algorithm is given by the steps in Table 5.3. The parse begins in state 0, which is a shift state, so the first token $($ is shifted onto the stack. Then, since the DFA indicates a transition from state 0 to state 3 on the symbol $($, state 3 is pushed onto the stack. State 3 is also a shift state, so the next $($ is shifted onto the stack, and the transition on $($ returns to state 3. Shifting again places a on the stack, and the transition on a from state 3 goes to state 2. We are now at step 4 in Table 5.3, and we have reached the first reduce state. Here state 2 and symbol a are popped from the stack, backing up to state 3 in the process. A is then pushed onto the stack and the A transition from state 3 to state 4 is taken. State 4 is a shift state, so $)$ is shifted onto the stack and the transition on $)$ takes the parse to state 5. Here a reduction by the rule $A \rightarrow (A)$ occurs, popping states 5, 4, 3, and symbols $)$, A , $($ from the stack. The parse is now in state 3, and again A and state 4 are pushed onto the stack. Again, $)$ is shifted onto the stack, and state 5 is pushed. Another reduction by $A \rightarrow (A)$ removes the string $(3 A 4) 5$ (backwards) from the stack, leaving the parse in state 0. Now A is pushed and the A transition from state 0 to state 1 is taken. State 1 is the accepting state. Since the input is now empty, the parse algorithm accepts.

Figure 5.5
The DFA of sets of LR(0)
items for Example 5.9

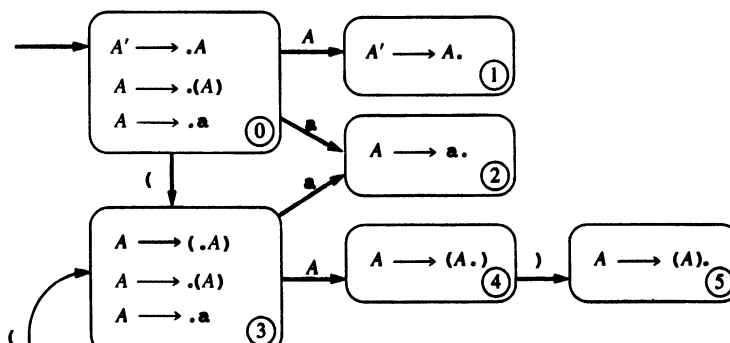


Table 5.3

Parsing actions for Example 5.9		Parsing stack	Input	Action
1		\$ 0	((a)) \$	shift
2		\$ 0(3	(a)) \$	shift
3		\$ 0(3(3	a)) \$	shift
4		\$ 0(3(3 a 2	)) \$	reduce $A \rightarrow a$
5		\$ 0(3(3 A 4	)) \$	shift
6		\$ 0(3(3 A 4) 5	) \$	reduce $A \rightarrow (A)$
7		\$ 0(3 A 4	) \$	shift
8		\$ 0(3 A 4) 5	\$	reduce $A \rightarrow (A)$
9		\$ 0 A 1	\$	accept

§

The DFA of sets of items and the actions specified by the LR(0) parsing algorithm can be combined into a parsing table, so that LR(0) parsing becomes a table-driven parsing method. A typical organization is for the table rows to be labeled with the states of the DFA, and the columns to be labeled as follows. Since LR(0) parsing states are either “shift” states or “reduce” states, one column is reserved to indicate this for each state. In the case of a “reduce” state, a further column is used to indicate the grammar rule choice to be used in the reduction. In the case of a shift state, the symbol to be shifted determines the next state (via the DFA), and so there must be a column for every token, whose entries are the new states to be entered on a shift of that token. Transitions on nonterminals (which are pushed during a reduction) represent a special case, since they are not actually seen in the input, although the parser acts as though they have been shifted. Thus, in a “shift” state there needs also to be a column for each nonterminal, and traditionally these columns are listed in a separate part of the table called the **goto** section.

An example of such a parsing table is Table 5.4, which is the table for the grammar of Example 5.9. The reader is encouraged to verify that this table leads to the parsing actions for that example as given in Table 5.3.

Table 5.4

Parsing table for the grammar of Example 5.9	State	Action	Rule	Input			Goto
				(	a	)	A
0	shift			3	2		1
1	reduce	$A' \rightarrow A$					
2	reduce	$A \rightarrow a$					
3	shift			3	2		4
4	shift					5	
5	reduce	$A \rightarrow (A)$					

We note that empty entries in such a parsing table represent errors. In practical situations where error recovery is necessary, we will need to precisely specify what

action the parser takes for each of these blank entries. We delay this discussion until a later section.

5.3 SLR(1) PARSING

5.3.1 The SLR(1) Parsing Algorithm

Simple LR(1), or SLR(1), parsing uses the DFA of sets of LR(0) items as constructed in the previous section. It increases the power of LR(0) parsing significantly, however, by using the next token in the input string to direct its actions. It does so in two ways. First, it consults the input token *before* a shift to make sure that an appropriate DFA transition exists. Second, it uses the Follow set of a nonterminal, as constructed in Section 4.3, to decide if a reduction should be performed. Surprisingly, this simple use of lookahead is powerful enough to parse almost all commonly occurring language constructs.

Definition

The SLR(1) parsing algorithm. Let s be the current state (at the top of the parsing stack). Then actions are defined as follows:

1. If state s contains any item of the form $A \rightarrow \alpha.X\beta$, where X is a terminal, and X is the next token in the input string, then the action is to shift the current input token onto the stack, and the new state to be pushed on the stack is the state containing the item $A \rightarrow \alpha.X.\beta$.
 2. If state s contains the complete item $A \rightarrow \gamma.$, and the next token in the input string is in $\text{Follow}(A)$, then the action is to reduce by the rule $A \rightarrow \gamma$. A reduction by the rule $S' \rightarrow S$, where S is the start state, is equivalent to acceptance; this will happen only if the next input token is $\$$.⁴ In all other cases, the new state is computed as follows. Remove the string γ and all of its corresponding states from the parsing stack. Correspondingly, back up in the DFA to the state from which the construction of γ began. By construction, this state must contain an item of the form $B \rightarrow \alpha.A\beta$. Push A onto the stack, and push the state containing the item $B \rightarrow \alpha.A.\beta$.
 3. If the next input token is such that neither of the above two cases applies, an error is declared.
-

We say that a grammar is an **SLR(1) grammar** if the application of the above SLR(1) parsing rules results in no ambiguity. In particular, a grammar is SLR(1) if and only if, for any state s , the following two conditions are satisfied:

4. Indeed, the Follow set for the augmented start state S' of any grammar is always the set consisting only of $\$$, since S' appears only in the grammar rule $S' \rightarrow S$.

1. For any item $A \rightarrow \alpha.X\beta$ in s with X a terminal, there is no complete item $B \rightarrow \gamma.$ in s with X in $\text{Follow}(B)$.
2. For any two complete items $A \rightarrow \alpha.$ and $B \rightarrow \beta.$ in s , $\text{Follow}(A) \cap \text{Follow}(B)$ is empty.

A violation of the first of these conditions represents a **shift-reduce conflict**. A violation of the second of these conditions represents a **reduce-reduce conflict**.

These two conditions are similar in spirit to the two conditions for LL(1) parsing stated in the previous chapter, except that, as with all shift-reduce parsing methods, decisions on which grammar rule to use can be delayed until the last possible moment, resulting in a more powerful parser.

A parsing table for SLR(1) parsing can also be constructed in a manner similar to that for LR(0) parsing described in the previous section. The differences are as follows. Since a state can have both shifts and reduces in an SLR(1) parser (depending on the lookahead), each entry in the input section must now have a “shift” or “reduce” label, and grammar rule choices must also be placed in entries marked “reduce.” This also makes the action and rule columns unnecessary. Since the end of input symbol \$ can also be a legal lookahead, a new column must be created for this symbol in the input section. We demonstrate the construction of the SLR(1) parsing table along with our first example of SLR(1) parsing.

Example 5.10

Consider the grammar of Example 5.8, whose DFA of sets of items is given in Figure 5.4. As we stated, this grammar is not LR(0), but it is SLR(1). The Follow sets for the nonterminals are $\text{Follow}(E') = \{\$ \}$ and $\text{Follow}(E) = \{\$, +\}$. The SLR(1) parsing table is given in Table 5.5. In the table a shift is indicated by the letter s in the entry, and a reduce by the letter r. Thus, in state 1 on input +, a shift is indicated, together with a transition to state 3. In state 2 on input +, on the other hand, a reduction by production $E \rightarrow n$ is indicated. We have also written the action “accept” in state 1 on input \$ instead of r ($E' \rightarrow E$).

Table 5.5

SLR(1) parsing table for
Example 5.10

State	Input			Goto
	n	+	\$	
0	s2			1
1		s3	accept	
2		r ($E \rightarrow n$)	r ($E \rightarrow n$)	
3	s4			
4		r ($E \rightarrow E + n$)	r ($E \rightarrow E + n$)	

We finish this example with a parse of the string $n + n + n$. The parsing steps are given in Table 5.6. Step 1 in this figure begins in state 0 on input token n , and the parsing table indicates the action “s2,” that is, shift the token onto the stack and go into state 2. We indicate this in Table 5.6 with the phrase “shift 2.” In step 2 of the figure, the

parser is in state 2 with input token $+$, and the table indicates a reduction by the rule $E \rightarrow n$. In this case state 2 and token n are popped from the stack, exposing state 0. The symbol E is pushed and the Goto for E is taken from state 0 to state 1. In step 3 the parser is in state 1 with input token $+$, and the table indicates a shift and a transition to state 3. In state 3 on input n the table also indicates a shift and a transition to state 4. In state 4 on input $+$ the table indicates a reduction by the rule $E \rightarrow E + n$. This reduction is accomplished by popping the string $E + n$ and its associated states from the stack, exposing state 0 again, pushing E and taking the Goto to state 1. The remaining steps in the parse are similar.

Table 5.6

Parsing actions for
Example 5.10

	Parsing stack	Input	Action
1	\$ 0	$n + n + n \$$	shift 2
2	\$ 0 n 2	$+ n + n \$$	reduce $E \rightarrow n$
3	\$ 0 E 1	$+ n + n \$$	shift 3
4	\$ 0 E 1 + 3	$n + n \$$	shift 4
5	\$ 0 E 1 + 3 n 4	$+ n \$$	reduce $E \rightarrow E + n$
6	\$ 0 E 1	$+ n \$$	shift 3
7	\$ 0 E 1 + 3	$n \$$	shift 4
8	\$ 0 E 1 + 3 n 4	$\$$	reduce $E \rightarrow E + n$
9	\$ 0 E 1	$\$$	accept

§

Example 5.11

Consider the grammar of balanced parentheses, whose DFA of LR(0) items is given in Figure 5.3 (page 205). A straightforward computation yields $\text{Follow}(S') = \{\$ \}$ and $\text{Follow}(S) = \{\$,) \}$. The SLR(1) parsing table is given in Table 5.7. Note how the non-LR(0) states 0, 2, and 4 have both shifts and reductions by the ϵ -production $S \rightarrow \epsilon$. Table 5.8 gives the steps that the SLR(1) parsing algorithm takes to parse the string $() ()$. Note how the stack continues to grow until the final reductions. This is characteristic of bottom-up parsers in the presence of right-recursive rules such as $S \rightarrow (S) S$. Thus, right recursion can cause stack overflow, and so is to be avoided if possible.

Table 5.7

SLR(1) parsing table for
Example 5.11

State	Input			Goto
	(	)	\$	
0	s2	$r(S \rightarrow \epsilon)$	$r(S \rightarrow \epsilon)$	1
1			accept	
2	s2	$r(S \rightarrow \epsilon)$	$r(S \rightarrow \epsilon)$	3
3		s4		
4	s2	$r(S \rightarrow \epsilon)$	$r(S \rightarrow \epsilon)$	5
5		$r(S \rightarrow (S) S)$	$r(S \rightarrow (S) S)$	

Table 5.8

Parsing actions for
Example 5.11

	Parsing stack	Input	Action
1	\$ 0	() ()\$	shift 2
2	\$ 0 (2	) ()\$	reduce $S \rightarrow \varepsilon$
3	\$ 0 (2 S 3	) ()\$	shift 4
4	\$ 0 (2 S 3) 4	()\$	shift 2
5	\$ 0 (2 S 3) 4 (2	)\$	reduce $S \rightarrow \varepsilon$
6	\$ 0 (2 S 3) 4 (2 S 3	)\$	shift 4
7	\$ 0 (2 S 3) 4 (2 S 3) 4	\$	reduce $S \rightarrow \varepsilon$
8	\$ 0 (2 S 3) 4 (2 S 3) 4 S 5	\$	reduce $S \rightarrow (S) S$
9	\$ 0 (2 S 3) 4 S 5	\$	reduce $S \rightarrow (S) S$
10	\$ 0 S 1	\$	accept

§

5.3.2 Disambiguating Rules for Parsing Conflicts

Parsing conflicts in SLR(1) parsing, as with all shift-reduce parsing methods, can be of two kinds: shift-reduce conflicts and reduce-reduce conflicts. In the case of shift-reduce conflicts, there is a natural disambiguating rule, which is to always prefer the shift over the reduce. Most shift-reduce parsers therefore automatically resolve shift-reduce conflicts by preferring the shift over the reduce. The case of reduce-reduce conflicts is more difficult; such conflicts often (but not always) indicate an error in the design of the grammar. (Examples of such conflicts will be given later.) Preferring the shift over the reduce in a shift-reduce conflict automatically incorporates the most closely nested rule for the dangling else ambiguity in if-statements, as the following example shows. This is one reason for letting the ambiguity remain in a programming language grammar.

Example 5.12

Consider the grammar of simplified if statements used in the previous chapters (see, e.g., Chapter 3, Section 3.4.3):

$$\begin{aligned}
 \text{statement} &\rightarrow \text{if-stmt} \mid \text{other} \\
 \text{if-stmt} &\rightarrow \text{if (exp) statement} \\
 &\quad \mid \text{if (exp) statement else statement} \\
 \text{exp} &\rightarrow 0 \mid 1
 \end{aligned}$$

Since this is an ambiguous grammar, there must be a parsing conflict somewhere in any parsing algorithm. To see this in an SLR(1) parser, we simplify the grammar even further, to make the construction of the DFA of sets of items more manageable. We even eliminate the test expression altogether and write the grammar as follows (it still contains the dangling else ambiguity):

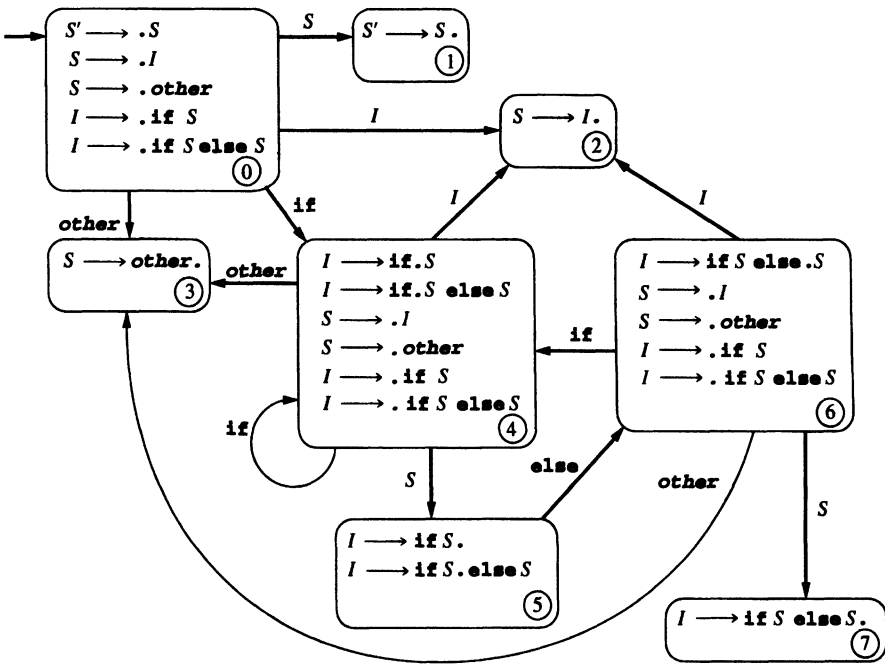
$$\begin{aligned}
 S &\rightarrow I \mid \text{other} \\
 I &\rightarrow \text{if } S \mid \text{if } S \text{ else } S
 \end{aligned}$$

The DFA of sets of items is shown in Figure 5.6. To construct the SLR(1) parsing actions we need the Follow sets for S and I . These are

$$\text{Follow}(S) = \text{Follow}(I) = \{\$, \text{else}\}$$

Now we can see the parsing conflict caused by the dangling else problem. It occurs in state 5 of the DFA, where the complete item $I \rightarrow \text{if } S$ indicates that a reduction by the rule $I \rightarrow \text{if } S$ is to take place on inputs **else** and **\$**, while the item $I \rightarrow \text{if } S \text{ else } S$ indicates that a shift of the input token is to take place on **else**. Thus, the dangling else will result in a shift-reduce conflict in the SLR(1) parsing table. Clearly, a disambiguating rule that prefers the shift over the reduce will remove the conflict and parse according to the most closely nested rule. (Indeed, if the reduction were preferred over the shift, there would be no way of entering states 6 or 7 in the DFA, which would result in spurious parsing errors.)

Figure 5.6
DFA of sets of LR(0) items
for Example 5.12



The SLR(1) parsing table that results from this grammar is shown in Table 5.9. In that table we have used a numbering scheme for the grammar rule choices in the reduce actions instead of writing out the rules themselves. The numbering is

- (1) $S \rightarrow I$
- (2) $S \rightarrow \text{other}$
- (3) $I \rightarrow \text{if } S$
- (4) $I \rightarrow \text{if } S \text{ else } S$

Note that it is not necessary to number the augmentation production $S' \rightarrow S$, since a reduction by this rule corresponds to acceptance and is written as “accept” in the table.

We warn the reader that the production numbers used in reduce entries are easy to confuse with state numbers used in shift and Goto entries. For example, in the table of Table 5.9, in state 5 the entry under input **else** is s6, indicating a shift and a transition to state 6, while the entry under input **\$** is r3, indicating a reduction by production number 3 (that is, $I \rightarrow \text{if } S$).

In Table 5.9 we have also removed the shift-reduce conflict in the table in favor of the shift. We have shaded the entry in the table to show where the conflict would have occurred.

Table 5.9

SLR(1) parsing table for
Example 5.12 (with
parsing conflict removed)

State	Input				Goto	
	if	else	other	\$	S	I
0	s4		s3		1	2
1				accept		
2		r1		r1		
3		r2		r2		
4	s4		s3		5	2
5		s6		r3		
6	s4		s3		7	2
7		r4		r4		

§

5.3.3 Limits of SLR(1) Parsing Power

SLR(1) parsing is a simple, effective extension of LR(0) parsing that is powerful enough to handle almost all practical language structures. Unfortunately, there are a few situations in which SLR(1) parsing is not quite powerful enough, and this will lead us to study the more powerful general LR(1) and LALR(1) parsing. The next example is a typical situation in which SLR(1) parsing fails.

Example 5.13

Consider the following grammar rules for statements, extracted and simplified from Pascal (a similar situation occurs in C):

```

stmt → call-stmt | assign-stmt
call-stmt → identifier
assign-stmt → var := exp
var → var [ exp ] | identifier
exp → var | number

```

This grammar models statements which can be either calls to parameterless procedures, or assignments of expressions to variables. Note that both assignments and procedure calls begin with an identifier. It is not until either the end of the statement or the token **:=** is seen that a parser can decide whether the statement is an assignment or a call. We

simplify this situation to the following grammar, where we have removed the alternative choices for a variable, and simplified the statement options, without changing the basic situation:

$$\begin{aligned} S &\rightarrow id \mid V := E \\ V &\rightarrow id \\ E &\rightarrow V \mid n \end{aligned}$$

To show how this grammar results in a parsing conflict in SLR(1) parsing, consider the start state of the DFA of sets of items:

$$\begin{aligned} S' &\rightarrow \cdot S \\ S &\rightarrow \cdot id \\ S &\rightarrow \cdot V := E \\ V &\rightarrow \cdot id \end{aligned}$$

This state has a shift transition on id to the state

$$\begin{aligned} S &\rightarrow id \cdot \\ V &\rightarrow id \cdot \end{aligned}$$

Now, $\text{Follow}(S) = \{\$, \}$ and $\text{Follow}(V) = \{:=, \$\}$ ($:=$ because of the rule $S \rightarrow V := E$, and $\$$ because an E can be a V). Thus, the SLR(1) parsing algorithm calls for a reduction in this state by both the rule $S \rightarrow id$ and the rule $V \rightarrow id$ under input symbol $\$$. (This is a reduce-reduce conflict.) This parsing conflict is actually a “phony” problem caused by the weakness of the SLR(1) method. Indeed, the reduction by $V \rightarrow id$ should *never* be made in this state when the input is $\$$, since a variable can never occur at the end of a statement until after the token $:=$ is seen and shifted. §

In the following two sections we show how this parsing problem can be removed by the use of more powerful parsing methods.

5.3.4 SLR(k) Grammars

As with other parsing algorithms, the SLR(1) parsing algorithm can be extended to SLR(k) parsing where parsing actions are based on $k \geq 1$ symbols of lookahead. Using the sets First_k and Follow_k as defined in the previous chapter, an SLR(k) parser uses the following two rules:

1. If state s contains an item of the form $A \rightarrow \alpha.X\beta$ (X a token), and $Xw \in \text{First}_k(X\beta)$ are the next k tokens in the input string, then the action is to shift the current input token onto the stack, and the new state to be pushed on the stack is the state containing the item $A \rightarrow \alpha.X\beta$.
2. If state s contains the complete item $A \rightarrow \alpha.$, and $w \in \text{Follow}_k(A)$ are the next k tokens in the input string, then the action is to reduce by the rule $A \rightarrow \alpha$.

SLR(k) parsing is more powerful than SLR(1) parsing when $k > 1$, but at a substantial cost in complexity, since the parsing table grows exponentially in size with k .

Typical language constructs that are not SLR(1) are handled better by using an LALR(1) parser, by using standard disambiguating rules, or by rewriting the grammar. While it is true that the simple non-SLR(1) grammar of Example 5.13 does happen to be SLR(2), the programming language problem that it came from is not SLR(k) for any k .

5.4 GENERAL LR(1) AND LALR(1) PARSING

In this section we study the most general form of LR(1) parsing, sometimes called **canonical** LR(1) parsing. This method overcomes the problem with SLR(1) parsing described at the end of the previous section, but at a cost of substantially increased complexity. Indeed, general LR(1) parsing is usually considered too complex to use in the construction of parsers in most situations. Fortunately, a modification of general LR(1) parsing, called LALR(1) (for “lookahead” LR parsing), retains most of the benefit of general LR(1) parsing, while preserving the efficiency of the SLR(1) method. The LALR(1) method has become the method of choice for parser generators such as Yacc, and we study it later in this section. However, to understand that method we must first study the general method.

5.4.1 Finite Automata of LR(1) Items

The difficulty with the SLR(1) method is that it applies lookaheads *after* the construction of the DFA of LR(0) items, a construction that ignores lookaheads. The power of the general LR(1) method is that it uses a new DFA that has the lookaheads built into its construction from the start. This DFA uses items that are an extension of LR(0) items. They are called **LR(1) items** because they include a single lookahead token in each item. More precisely, an LR(1) item is a pair consisting of an LR(0) item and a lookahead token. We write LR(1) items using square brackets as

$$[A \rightarrow \alpha.\beta, a]$$

where $A \rightarrow \alpha.\beta$ is an LR(0) item and a is a token (the lookahead).

To complete the definition of the automaton used for general LR(1) parsing, we need to define the transitions between LR(1) items. These are similar to the LR(0) transitions except that they also keep track of lookaheads. As with LR(0) items they include ϵ -transitions, and it is necessary to build a DFA whose states are sets of items that are ϵ -closures. The major difference between the LR(0) and LR(1) automata comes in the definition of the ϵ -transitions. We give the definition of the easier case (the non- ϵ -transitions) first, which are essentially identical to those of the LR(0) case.

Definition

Definition of LR(1) transitions (part 1). Given an LR(1) item $[A \rightarrow \alpha.X\gamma, a]$, where X is any symbol (terminal or nonterminal), there is a transition on X to the item $[A \rightarrow \alpha.X\gamma, a]$.

Note that in this case the same lookahead a appears in both items. Thus, these transitions do not cause new lookaheads to appear. Only ε -transitions “create” new lookaheads, as follows.

Definition

Definition of LR(1) transitions (part 2). Given an LR(1) item $[A \rightarrow \alpha.B \gamma, a]$, where B is a nonterminal, there are ε -transitions to items $[B \rightarrow \cdot\beta, b]$ for every production $B \rightarrow \beta$ and for every token b in $\text{First}(\gamma a)$.

Note how these ε -transitions keep track of the context in which the structure B needs to be recognized. Indeed, the item $[A \rightarrow \alpha.B \gamma, a]$ says that at this point in the parse we may want to recognize a B , but *only* if this B is followed by a string derivable from the string γa , and such strings must begin with a token in $\text{First}(\gamma a)$. Since the string γ follows B in the production $A \rightarrow \alpha B \gamma$, if a is constructed to be in $\text{Follow}(A)$, then $\text{First}(\gamma a) \subset \text{Follow}(B)$, and the b 's in the items $[B \rightarrow \cdot\beta, b]$ will always be in $\text{Follow}(B)$. The power of the general LR(1) method lies in the fact that the set $\text{First}(\gamma a)$ may be a *proper* subset of $\text{Follow}(B)$. (An SLR(1) parser essentially takes the lookaheads b from the entire Follow set.) Note also that the original lookahead a appears as one of the b 's only if γ can derive the empty string. Much of the time (especially in practical situations), this will only occur if γ is itself ε , in which case we get the special case of an ε -transition from $[A \rightarrow \alpha.B, a]$ to $[B \rightarrow \cdot\beta, a]$.

To complete the description of the construction of the DFA of sets of LR(1) items, it remains to specify the start state. We do this as in the LR(0) case by augmenting the grammar with a new start symbol S' and a new production $S' \rightarrow S$ (where S is the original start symbol). Then the start symbol of the NFA of LR(1) items becomes the item $[S' \rightarrow \cdot S, \$]$, where the $\$$ represents the end marker (and is the only symbol in $\text{Follow}(S')$). In effect, this says that we start by recognizing a string derivable from S , followed by the $\$$ symbol.

We now look at several examples of the construction of the DFA of LR(1) items.

Example 5.14

Consider the grammar

$$A \rightarrow (A) \mid a$$

of Example 5.9. We begin the construction of its DFA of sets of LR(1) items by augmenting the grammar and forming the initial LR(1) item $[A' \rightarrow \cdot A, \$]$. The ε -closure of this item is the start state of the DFA. Since A is not followed by any symbols in this item (in the terminology of the previous discussion of transitions, the string γ is ε), there are ε -transitions to items $[A \rightarrow \cdot (A), \$]$ and $[A \rightarrow \cdot a, \$]$ (that is, $\text{First}(\gamma \$) = \{ \$ \}$, in the terminology of the previous discussion). The start state (state 0) is then the set of these three items:

$$\begin{array}{ll} \text{State 0:} & [A' \rightarrow \cdot A, \$] \\ & [A \rightarrow \cdot (A), \$] \\ & [A \rightarrow \cdot a, \$] \end{array}$$

From this state there is a transition on A to the closure of the set containing the item $[A' \rightarrow A., \$]$, and since there are no transitions from complete items, this state contains just the single item $[A' \rightarrow A., \$]$. There are no transitions out of this state, and we will number it state 1:

State 1: $[A' \rightarrow A., \$]$

(This is the state from which the LR(1) parsing algorithm will generate accept actions.)

Returning to state 0, there is also a transition on the token $($ to the closure of the set consisting of the item $[A \rightarrow (. A), \$]$. Since there are ϵ -transitions from this item, this closure is nontrivial. Indeed, there are ϵ -transitions from this item to the items $[A \rightarrow . (A),)]$ and $[A \rightarrow . a,)]$. This is because, in the item $[A \rightarrow (. A), \$]$, A is being recognized in the right-hand side *in the context of parentheses*. That is, the Follow of the right-hand A is $\text{First}() \$) = \{) \}$. Thus, we have obtained a new lookahead token in this situation, and the new DFA state consists of the following items:

State 2: $[A \rightarrow (. A), \$]$
 $[A \rightarrow . (A),)]$
 $[A \rightarrow . a,)]$

We return once more to state 0, where we find a last transition to the state generated by the item $[A \rightarrow a., \$]$. Since this is a complete item, it is a state of one item:

State 3: $[A \rightarrow a., \$]$

Now we return to state 2. From this state there is a transition on A to the ϵ -closure of $[A \rightarrow (A .), \$]$, which is a state with one item:

State 4: $[A \rightarrow (A .), \$]$

There is also a transition on $($ to the ϵ -closure of $[A \rightarrow (. A),)]$. Here, we also generate the closure items $[A \rightarrow . (A),)]$ and $[A \rightarrow . a,)]$, for the same reasons as in the construction of state 2. Thus, we obtain the new state:

State 5: $[A \rightarrow (. A),)]$
 $[A \rightarrow . (A),)]$
 $[A \rightarrow . a,)]$

Note that this state is the same as state 2, except for the lookahead of the first item.

Finally, we have a transition on the token a from state 2 to state 6:

State 6: $[A \rightarrow a.,)]$

Note again that this is almost the same as state 3, differing from it only in the lookahead.

The next state that has a transition is state 4, which has a transition on the token $)$ to the state

Returning to state 5, we have a transition from this state to itself on (, a transition on A to the state

State 8: $[A \rightarrow (A \cdot),)]$

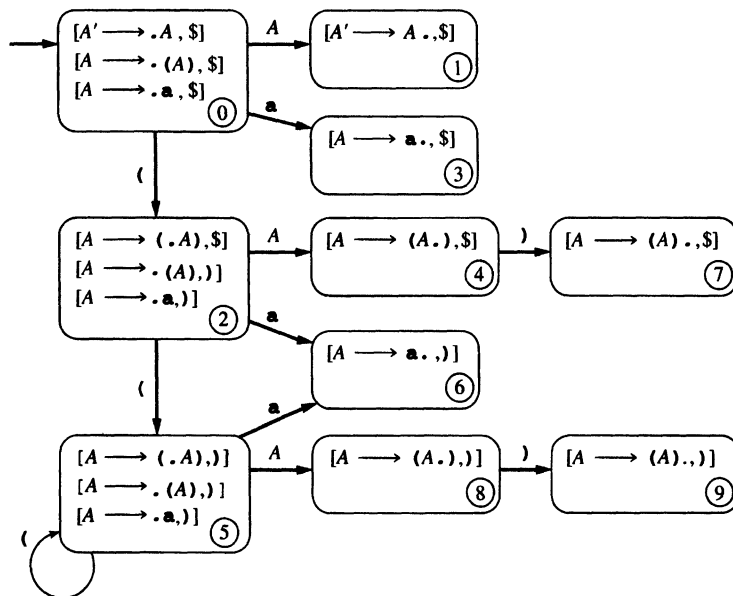
and a transition to the already constructed state 6 on a.

Finally, there is a transition on) from state 8 to

State 9: $[A \rightarrow (A) \cdot,)]$.

Thus, the DFA of LR(1) items for this grammar has ten states. The complete DFA is shown in Figure 5.7. Comparing this with the DFA of sets of LR(0) items for the same grammar (see Figure 5.5, page 208), we see that the DFA of LR(1) items is almost twice as large. This is not unusual. Indeed, the number of LR(1) states can exceed the number of LR(0) states by a factor of 10 in complex situations with many lookahead tokens.

Figure 5.7
The DFA of sets of LR(1)
items for Example 5.14



§

5.4.2 The LR(1) Parsing Algorithm

Before considering additional examples, we need to complete the discussion of general LR(1) parsing by restating the parsing algorithm based on the new DFA construction. This is easy to do, since it is just a restatement of the SLR(1) parsing algorithm, except that it uses the lookahead tokens in the LR(1) items instead of the Follow sets.

The General LR(1) parsing algorithm Let s be the current state (at the top of the parsing stack). Then actions are defined as follows:

1. If state s contains any LR(1) item of the form $[A \rightarrow \alpha.X\beta, a]$, where X is a terminal, and X is the next token in the input string, then the action is to shift the current input token onto the stack, and the new state to be pushed on the stack is the state containing the LR(1) item $[A \rightarrow \alpha.X\beta, a]$.
2. If state s contains the complete LR(1) item $[A \rightarrow \alpha., a]$, and the next token in the input string is a , then the action is to reduce by the rule $A \rightarrow \alpha$. A reduction by the rule $S' \rightarrow S$, where S is the start state, is equivalent to acceptance. (This will happen only if the next input token is $\$$.) In the other cases, the new state is computed as follows. Remove the string α and all of its corresponding states from the parsing stack. Correspondingly, back up in the DFA to the state from which the construction of α began. By construction, this state must contain an LR(1) item of the form $[B \rightarrow \alpha.A\beta, b]$. Push A onto the stack, and push the state containing the item $[B \rightarrow \alpha.A\beta, b]$.
3. If the next input token is such that neither of the above two cases applies, an error is declared.

As with the previous methods, we say that a grammar is an **LR(1) grammar** if the application of the above general LR(1) parsing rules results in no ambiguity. In particular, a grammar is LR(1) if and only if, for any state s , the following two conditions are satisfied:

1. For any item $[A \rightarrow \alpha.X\beta, a]$ in s with X a terminal, there is no item in s of the form $[B \rightarrow \gamma., X]$ (otherwise there is a shift-reduce conflict).
2. There are no two items in s of the form $[A \rightarrow \alpha., a]$ and $[B \rightarrow \beta., a]$ (otherwise, there is a reduce-reduce conflict).

We remark also that a parsing table can be constructed from the DFA of sets of LR(1) items that expresses the general LR(1) parsing algorithm. This table has exactly the same form as the table for SLR(1) parsers, as the next example shows.

Example 5.15

We give the general LR(1) parsing table for the grammar of Example 5.14 in Table 5.10. It can be easily constructed from the DFA of Figure 5.7. In the table we have used the following numbering for the grammar rule choices in the reduction actions:

- (1) $A \rightarrow (A)$
- (2) $A \rightarrow \mathbf{a}$

Thus, the entry r_2 in state 3 with lookahead $\$$ indicates a reduction by the rule $A \rightarrow \mathbf{a}$.

Since the steps in the parse of a particular string in general LR(1) parsing are just as they would be in SLR(1) or LR(0) parsing, we omit an example of such a parse. In

Table 5.10

General LR(1) parsing table
for Example 5.14

State	Input				Goto
	(	a	)	\$	
0	s2	s3			1
1				accept	
2	s5	s6			4
3				r2	
4			s7		
5	s5	s6			8
6			r2		
7				r1	
8			s9		
9			r1		

§

subsequent examples in this section, we will also omit the explicit construction of the parsing table, since it is so easily supplied from the DFA.

In practice, almost all reasonable programming language grammars are LR(1), unless they are ambiguous. It is, of course, possible to construct examples of nonambiguous grammars that fail to be LR(1), but we shall not do so here (see the Exercises). Such examples tend to be contrived and can usually be avoided in practical situations. In fact, a programming language rarely even needs the power that general LR(1) parsing provides. Indeed, the example we have used to introduce LR(1) parsing (Example 5.14) is already an LR(0) grammar (and, hence, also SLR(1)).

The next example shows that general LR(1) parsing solves the lookahead problem for the grammar of Example 5.13 that failed to be SLR(1).

Example 5.16

The grammar of Example 5.13 in simplified form is as follows:

$$\begin{aligned}
 S &\rightarrow id \mid V := E \\
 V &\rightarrow id \\
 E &\rightarrow V \mid \epsilon
 \end{aligned}$$

We construct the DFA of sets of LR(1) items for this grammar. The start state is the closure of the item $[S' \rightarrow .S, \$]$. It contains the items $[S \rightarrow .id, \$]$ and $[S \rightarrow .V := E, \$]$. This last item also gives rise to the closure item $[V \rightarrow .id, :=]$, since $S \rightarrow .V := E$ indicates that a V may be recognized, but only if it is followed by an assignment token. Thus, the token $:=$ appears as the lookahead of the initial item $V \rightarrow .id$. Summarizing, the start state consists of the following LR(1) items:

$$\begin{aligned}
 \text{State 0: } &[S' \rightarrow .S, \$] \\
 &[S \rightarrow .id, \$] \\
 &[S \rightarrow .V := E, \$] \\
 &[V \rightarrow .id, :=]
 \end{aligned}$$

From this state there is a transition on S to the state of one item

State 1: $[S' \rightarrow S., \$]$

and a transition on id to the state of two items

State 2: $[S \rightarrow id., \$]$
 $[V \rightarrow id., :=]$

There is also a transition on V from state 0 to the state of one item

State 3: $[S \rightarrow V. := E, \$]$

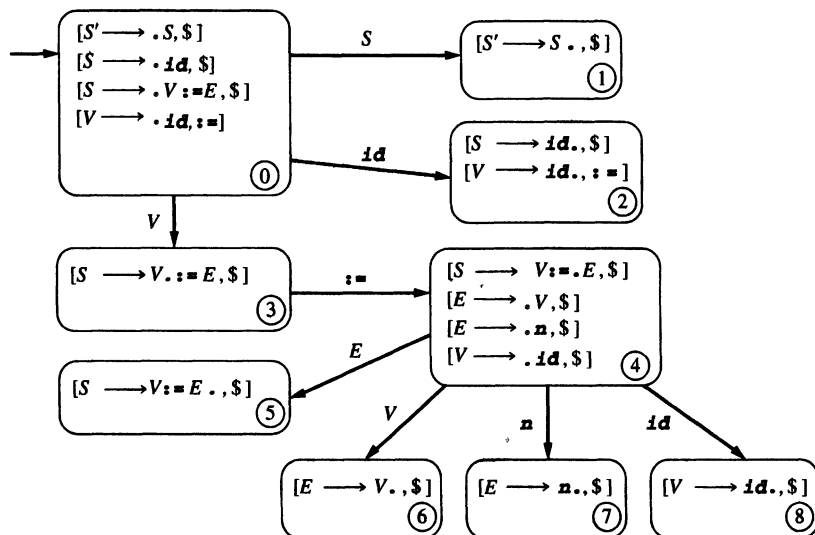
From states 1 and 2 there are no transitions, but there is a transition from state 3 on $:=$ to the closure of item $[S \rightarrow V :=. E, \$]$. Since E has no symbols following it in this item, this closure includes the items $[E \rightarrow .V, \$]$ and $[E \rightarrow .n, \$]$. Finally, the item $[E \rightarrow .V, \$]$ leads to the closure item $[V \rightarrow .id, \$]$, since V also has no symbols following it in this case. (Contrast this to the situation in state 0, where a V was followed by an assignment. Here V *cannot* be followed by an assignment, because an assignment token has already been seen.) The complete state is then

State 4: $[S \rightarrow V :=. E, \$]$
 $[E \rightarrow .V, \$]$
 $[E \rightarrow .n, \$]$
 $[V \rightarrow .id, \$]$

The remaining states and transitions are easy to construct, and we leave it to the reader to do so. The complete DFA of sets of LR(1) items is shown in Figure 5.8.

Now consider state 2. This was the state that gave rise to the SLR(1) parsing conflict. The LR(1) items clearly distinguish the two reductions by their lookaheads: reduce by $S \rightarrow id$ on $\$$ and by $V \rightarrow id$ on $:=$. Thus, this grammar is LR(1).

Figure 5.8
The DFA of sets of LR(1)
items for Example 5.16



5.43 LALR(1) Parsing

LALR(1) parsing is based on the observation that, in many cases, the size of the DFA of sets of LR(1) items is due in part to the existence of many different states that have the same set of first components in their items (the LR(0) items), while differing only in their second components (the lookahead symbols). For example, the DFA of LR(1) items of Figure 5.7 has 10 states, while the corresponding DFA of LR(0) items (Figure 5.5) has only 6. Indeed, in Figure 5.7 each state of the pairs of states 2 and 5, 4 and 8, 7 and 9, and 3 and 6 differs from the other only in the lookahead components of its items. Consider, for instance, states 2 and 5. These two states differ only in their first item, and only in the lookahead of that item: state 2 has the first item $[A \rightarrow (. A) , \$]$, with \$ as lookahead, while state 5 has the first item $[A \rightarrow (. A) ,)]$, with) as lookahead.

The LALR(1) parsing algorithm expresses the fact that it makes sense to identify all such states and combine their lookaheads. In doing so, we always must end up with a DFA that is identical to the DFA of LR(0) items, except that each state consists of items with sets of lookaheads. In the case of complete items these lookahead sets are often smaller than the corresponding Follow sets. Thus, LALR(1) parsing retains some of the benefit of LR(1) parsing over SLR(1) parsing, while preserving the smaller size of the DFA of LR(0) items.

Formally, the **core** of a state of the DFA of LR(1) items is the set of LR(0) items consisting of the first components of all LR(1) items in the state. Since the construction of the DFA of LR(1) items uses transitions that are the same as in the construction of the DFA of LR(0) items, except for their effect on the lookahead parts of the items, we obtain the following two facts, which form the basis for the LALR(1) parsing construction.

FIRST PRINCIPLE OF LALR(1) PARSING

The core of a state of the DFA of LR(1) items is a state of the DFA of LR(0) items.

SECOND PRINCIPLE OF LALR(1) PARSING

Given two states s_1 and s_2 of the DFA of LR(1) items that have the same core, suppose there is a transition on the symbol X from s_1 to a state t_1 . Then there is also a transition on X from state s_2 to a state t_2 , and the states t_1 and t_2 have the same core.

Taken together, these two principles allow us to construct the **DFA of LALR(1) items**, which is constructed from the DFA of LR(1) items by identifying all states that have the same core and forming the union of the lookahead symbols for each LR(0) item. Thus, each LALR(1) item in this DFA will have an LR(0) item as its first component and a set of lookahead tokens as its second component.⁵ In subsequent examples we will denote multiple lookaheads by writing a / between them. Thus, the LALR(1) item $[A \rightarrow \alpha.\beta, a / b / c]$ has a lookahead set consisting of the symbols a , b , and c .

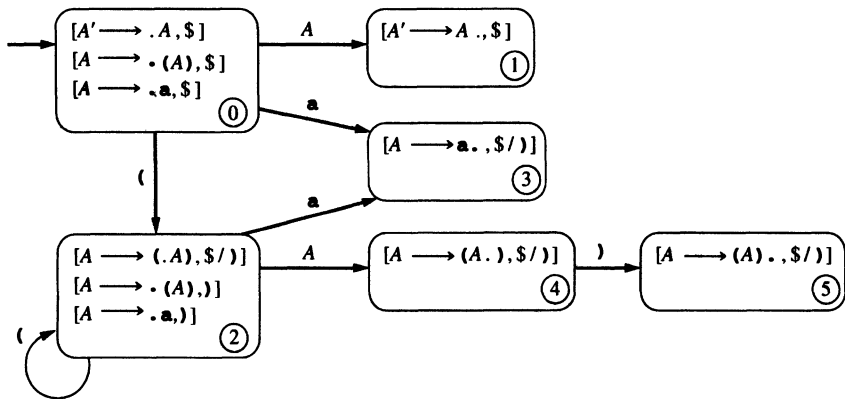
5. DFAs of LR(1) items could, in fact, also use sets of lookahead symbols to represent multiple items in the same state that share their first components, but we find it convenient to use this representation for the LALR(1) construction, where it is most appropriate.

We give an example to demonstrate this construction.

Example 5.17

Consider the grammar of Example 5.14, whose DFA of LR(1) items is given in Figure 5.7. Identifying states 2 and 5, 4 and 8, 7 and 9, and 3 and 6 gives the DFA of LALR(1) items in Figure 5.9. In that figure we have retained the numbering of the states 2, 3, 4, and 7, and added the lookaheads from states 5, 6, 8, and 9. As we expect, this DFA is identical to the DFA of LR(0) items (Figure 5.5), except for the lookaheads.

Figure 5.9
The DFA of sets of LALR(1)
items for Example 5.17



§

The algorithm for LALR(1) parsing using the condensed DFA of LALR(1) items is identical to the general LR(1) parsing algorithm described in the previous section. As before, we call a grammar an **LALR(1) grammar** if no parsing conflicts arise in the LALR(1) parsing algorithm. It is possible for the LALR(1) construction to create parsing conflicts that do not exist in general LR(1) parsing, but this rarely happens in practice. Indeed, if a grammar is LR(1), then the LALR(1) parsing table cannot have any shift-reduce conflicts; there may be reduce-reduce conflicts, however (see the Exercises). Nevertheless, if a grammar is SLR(1), then it certainly is LALR(1), and LALR(1) parsers often do as well as general LR(1) parsers in removing typical conflicts that occur in SLR(1) parsing. For example, the non-SLR(1) grammar of Example 5.16 is LALR(1): the DFA of LR(1) items of Figure 5.8 is also the DFA of LALR(1) items. If, as in this example, the grammar is already LALR(1), the only consequence of using LALR(1) parsing over general LR parsing is that, in the presence of errors, some spurious reductions may be made before error is declared. For example, we see from Figure 5.9 that, given the erroneous input string **a**), an LALR(1) parser will perform the reduction $A \rightarrow a$ before declaring error, while a general LR(1) parser will declare error immediately after a shift of the token **a**.

Combining LR(1) states to form the DFA of LALR(1) items solves the problem of large parsing tables, but it still requires the entire DFA of LR(1) items to be computed. In fact, it is possible to compute the DFA of LALR(1) items directly from the DFA of LR(0) items through a process of **propagating lookaheads**. Though we will not

describe this process formally, it is instructive to see how this can be done relatively easily.

Consider the LALR(1) DFA of Figure 5.9. We begin constructing lookaheads by adding the endmarker \$ to the lookahead of the augmentation item $A' \rightarrow .A$ in state 0. (The \$ lookahead is said to be **spontaneously generated**.) Then by the rules of ϵ -closure, the \$ propagates to the two closure items (the A on the right-hand side of the kernel item $A' \rightarrow .A$ is followed by the empty string). By following the three transitions from state 0, the \$ propagates to the kernel items of states 1, 3, and 2. Continuing with state 2, the closure items get the lookahead), again by spontaneous generation (because A on the right-hand side of the kernel item $A \rightarrow (.A)$ comes before a right parenthesis). Now the transition on a to state 3 causes the) to be propagated to the lookahead of the item in that state. Also, the transition on (from state 2 to itself causes the) to be propagated to the lookahead of the kernel item (this is why the kernel item has both \$ and) in its lookahead set). Now the lookahead set \$/) propagates to state 4 and then to state 7. Thus, through this process, we have obtained the DFA of LALR(1) items of Figure 5.9 directly from the DFA of LR(0) items.

5.5 Yacc: AN LALR(1) PARSER GENERATOR

A **parser generator** is a program that takes as its input a specification of the syntax of a language in some form, and produces as its output a parse procedure for that language. Historically, parser generators were called **compiler-compilers**, since traditionally all compilation steps were performed as actions included within the parser. The modern view is to consider the parser to be just one part of the compilation process, so this term is out of date. One widely used parser generator that incorporates the LALR(1) parsing algorithm is called **Yacc** (for yet another compiler-compiler). We will give an overview of Yacc in this section, and then in the following section we will use Yacc to develop a parser for the TINY language. Since there are a number of different implementations of Yacc, as well as several public domain versions commonly called **Bison**,⁶ there are numerous variations in the details of their operation, which may differ somewhat from the version presented here.⁷

5.5.1 Yacc Basics

Yacc takes a specification file (usually with a **.y** suffix) and produces an output file consisting of C source code for the parser (usually in a file called **y.tab.c** or **ytab.c** or, more recently **<filename>.tab.c**, where **<filename>.y** is the input file). A Yacc specification file has the basic format

6. One popular version, Gnu Bison, is part of the Gnu software distributed by the Free Software Foundation—see the Notes and References section.

7. Indeed, we have used several different versions to generate subsequent examples.

the header file generated by Yacc, containing the token definitions, must be included into most of the other code files, but cannot be included directly in the Yacc-generated parser, since that would repeat the internal definitions. The solution is to begin the Yacc declaration section with the definition of a flag **YYPARSER** (line 4007), which will be included in the Yacc parser and will indicate when the C compiler is inside the parser. We use that flag (lines 4226–4236) to selectively include the Yacc-generated header file **y.tab.h** into **globals.h**.

A second problem occurs with the **ENDFILE** token, which the scanner generates to indicate the end of the input file. Yacc assumes this token always has the value 0. We thus provide a direct definition of this token (line 4234) and include it in the selectively compiled section controlled by **YYPARSER**, since it is not needed internally by Yacc.

The final change to the **globals.h** file is to redefine **TokenType** to be a synonym for **int** (line 4252), since the Yacc tokens all have integer values. This avoids unnecessary replacements of the previous enumerated type **TokenType** in other files.

5.7 ERROR RECOVERY IN BOTTOM-UP PARSERS

5.7.1 Detecting Errors in Bottom-Up Parsing

A bottom-up parser will detect an error when a blank (or error) entry is detected in the parsing table. Obviously, it makes sense that errors should be detected as soon as possible, since error messages can be more meaningful and specific. Thus, a parsing table should have as many blank entries as possible.

Unfortunately, this goal conflicts with an equally important one: reducing the size of the parsing table. We have already seen (Table 5.11) that Yacc fills in as many table entries as it can with default reductions, so that a large number of reductions may take place on the parsing stack before an error is declared. This obscures the precise source of the error and may lead to uninformative error messages.

An additional feature of bottom-up parsing is that the power of the particular algorithm used can affect the ability of the parser to detect errors early. An LR(1) parser can, for example, detect errors earlier than an LALR(1) or SLR(1) parser, and these latter can detect errors earlier than an LR(0) parser. As a simple example of this, compare the LR(0) parsing table of Table 5.4 (page 209) with the LR(1) parsing table of Table 5.10 (page 222) for the same grammar. Given the incorrect input string (**a** \$, the LR(1) parsing table of Table 5.10 will shift (and **a** onto the stack and move to state 6. In state 6 there is no entry under \$, so an error will be reported. By contrast, the LR(0) algorithm (and the SLR(1) algorithm as well) will reduce by $A \rightarrow \mathbf{a}$ before discovering the lack of a balancing right parenthesis. Similarly, given the incorrect string (**a**) \$, the general LR(1) parser will shift **a** and then declare error from state 3 on right parenthesis, while the LR(0) parser will again reduce by $A \rightarrow \mathbf{a}$ before declaring error. Of course, any bottom-up parser will always report the error eventually, after perhaps a number of “erroneous” reductions. None of these parsers will ever shift a token on error.

5.7.2 Panic Mode Error Recovery

As in top-down parsing, it is possible to achieve reasonably good error recovery in bottom-up parsers by judiciously removing symbols from either the parsing stack or

the input or both. Similar to LL(1) parsers, there are three possible alternative actions that might be contemplated:

- 1. Pop a state from the stack.
- 2. Successively pop tokens from the input until a token is seen for which we can restart the parse.
- 3. Push a new state onto the stack.

One particularly effective method for choosing which of these actions to take when an error occurs is as follows:

- 1. Pop states from the parsing stack until a state is found with nonempty Goto entries.
- 2. If there is a legal action on the current input token from one of the Goto states, push that state onto the stack and restart the parse. If there are several such states, prefer a shift to a reduce. Among the reduce actions, prefer one whose associated nonterminal is least general.
- 3. If there is no legal action on the current input token from one of the Goto states, advance the input until there is a legal action or the end of the input is reached.

These rules have the effect of forcing the recognition of a construct which was in the process of being recognized when the error occurred, and restarting the parse immediately thereafter. Error recovery that uses these or similar rules could be called **panic mode** error recovery, since it is similar to the top-down panic mode described in Section 4.5.

Unfortunately, these rules can result in an infinite loop, since step 2 pushes new states onto the stack. In that case, there are several possible solutions. One is to insist on a shift action from a Goto state in step 2. This may be too restrictive, however. Another solution is, if the next legal move is a reduction, to set a flag that causes the parser to keep track of the sequence of states during the following reductions, and if the same state recurs, to pop stack states until the original state is removed at which the error occurred, and begin again with step 1. If, at any time, a shift action occurs, the parser resets the flag and continues with a normal parse.

Example 5.19

Consider the simple arithmetic expression grammar whose Yacc parsing table is given in Table 5.11 (page 235). Now consider the erroneous input **(2+*)**. The parse proceeds normally until the ***** is seen. At that point, panic mode would cause the following actions to take place on the parsing stack:

Parsing stack	Input	Action
...	...	...
\$ 0 (6 E 10 + 7	*)\$	error: push T, goto 11
\$ 0 (6 E 10 + 7 T 11	*)\$	shift 9
\$ 0 (6 E 10 + 7 T 11 * 9	)\$	error: push F, goto 13
\$ 0 (6 E 10 + 7 T 11 * 9 F 13	)\$	reduce T → T * F

At the first error, the parser is in state 7, which has legal Goto states 11 and 4. Since state 11 has a shift on the next input token `*`, that Goto is preferred, and the token is shifted. At that point the parser is in state 9, with a right parenthesis on the input. This is again an error. In state 9 there is a single Goto entry (to state 11), and state 11 does have a legal action on `)` (albeit a reduction). The parse then proceeds normally to its conclusion. §

5.7.3 Error Recovery in Yacc

An alternative to panic mode is to use so-called **error productions**. An error production is a production that contains the pseudotoken **error** on its right-hand side. An error production marks a context in which erroneous tokens can be deleted until appropriate synchronizing tokens are seen, whence the parse can be restarted. In effect, error productions allow the programmer to manually mark those nonterminals whose Goto entries are to be used for error recovery.

Error productions are the principal method available in Yacc for error recovery. The behavior of a Yacc parser in the presence of errors, and its handling of error productions, is as follows.

1. When the parser detects an error during a parse (that is, it encounters an empty entry in the parsing table), it pops states from the parsing stack until it reaches a state in which the **error** pseudotoken is a legal lookahead. The effect is to discard input to the left of the error, and to view the input as containing the **error** pseudotoken. If there are no error productions, then **error** is never a legal lookahead for a shift, and the parser stack will be emptied, aborting the parse at the first error. (This is the behavior of the parser generated by the Yacc input of Figure 5.10, page 228.)
2. Once the parser has found a state on the stack in which **error** is a legal lookahead, it continues in normal fashion with shifts and reduces. The effect is as though **error** were seen in the input, followed by the original lookahead (that is, the lookahead that caused the error). If desired, the Yacc macro **yyclearin** can be used to discard the token that caused the error, and to use the token following it as the next lookahead (after **error**).
3. If, after an error occurs, the parser discovers further errors, then the input tokens causing the errors are silently discarded, until three successive tokens are shifted legally onto the parsing stack. During this time the parser is said to be in an “error state.” This behavior is designed to avoid cascades of error messages caused by the same error. This can, however, result in large amounts of input being thrown away before the parser exits the error state (just as with panic mode). The compiler writer can override this behavior by using the Yacc macro **yyerrok** to remove the parser from the error state, so that no further input will be discarded without a new error recovery.

We describe a few simple examples of this behavior, based on the Yacc input of Figure 5.10.

Example 5.20

Consider the following replacement for the rule for *command* in Figure 5.10:

```

command : exp      { printf("%d\n", $1); }
          | error    { yyerror("incorrect expression"); }
          ;

```

Consider also the erroneous input **2++3**. The parse of this string proceeds normally until the erroneous second **+** is reached. At that point the parse is given by the following parsing stack and input (we use Table 5.11 to describe this, although the addition of the error production will actually result in a slightly different parse table):

PARSING STACK	INPUT
\$0 <i>exp</i> 2 + 7	+3\$

Now the parser enters the error “state” (generating an error message such as “syntax error”), and begins popping states from the stack, until state 0 is uncovered. At this point the error production for *command* provides that **error** is a legal lookahead, and it will be shifted onto the parsing stack, and immediately reduced to *command*, causing the associated action to be executed (which prints the message “incorrect expression”). The resulting situation is now as follows:

PARSING STACK	INPUT
\$0 <i>command</i> 1	+3\$

At this point the only legal lookahead is the end of input (indicated by \$ here, corresponding to the return of 0 by **yylex**), and the parser will delete the remaining input tokens **+3** before exiting (while still in the “error state”). Thus, the addition of the error production has essentially the same effect as the version in Figure 5.10, except that we are now able to supply our own error message.

An even better error mechanism than this would allow the user to reenter the line after the erroneous input. In this case, a synchronizing token is needed, and the end of line marker is the only sensible one. Thus, the scanner must be modified to return the newline character (instead of 0), and with this modification we may write (but see Exercise 5.32):

```

command : exp '\n'    { printf("%d\n", $1); exit(0); }
          | error '\n'
          { yyerrok;
            printf("reenter expression: "); }
          command
          ;

```

This code has the effect that, when an error occurs, the parser will skip all tokens up to a newline, when it will execute the action represented by **yyerrok** and the **printf** statement. Then it will attempt to recognize another *command*. The call to **yyerrok** is needed here to cancel the “error state” after the newline is seen, since otherwise if a new

error occurs right after the newline, Yacc will silently delete tokens until it finds a sequence of three correct tokens. §

Example 5.21

Consider what would happen if an error production were added to the Yacc definition of Figure 5.10 as follows:

```

factor      : NUMBER          { $$ = $1; }
              | '(' exp ')'      { $$ = $2; }
              | error { $$ = 0; }
              ;

```

Consider first the erroneous input **2++3** as in the previous example. (We continue to use Table 5.11, although the additional error production results in a slightly different table.) As before, the parser will reach the following point:

PARSING STACK	INPUT
\$0 <i>exp</i> 2 + 7	+3\$

Now the error production for *factor* will provide that **error** is a legal lookahead in state 7 and **error** will be immediately shifted onto the stack and reduced to *factor*, causing the value 0 to be returned. Now the parser has reached the following point:

PARSING STACK	INPUT
\$0 <i>exp</i> 2 + 7 <i>factor</i> 4	+3\$

This is a normal situation, and the parser will continue to execute normally to the end. The effect is to interpret the input as **2+0+3**—the 0 between the two + symbols is there because that is where the **error** pseudotoken is inserted, and by the action for the error production, **error** is viewed as equivalent to a factor with value 0.

Now consider the erroneous input **2 3** (that is, two numbers with a missing operator). Here the parser reaches the position

PARSING STACK	INPUT
\$0 <i>exp</i> 2	3\$

At this point (if the rule for *command* has not been changed), the parser will (erroneously) reduce by the rule *command* → *exp* (and print the value 2), even though a number is not a legal follow symbol for *command*. The parser then reaches the position

PARSING STACK	INPUT
\$0 <i>command</i> 1	3\$

Now an error is detected, and state 1 is popped from the parsing stack, uncovering state 0. At this point the error production for *factor* allows for **error** to be a legal lookahead from state 0, and **error** is shifted onto the parsing stack, resulting in another cascade of reductions and the printing of the value 0 (the value returned by the error production). Now the parser is back at the same position in the parse, with the number **3** still

in the input! Fortunately, the parser is already in the “error state” and will not shift **error** again, but throws away the number **3**, uncovering the correct lookahead for state 1, whereupon the parser exits.¹⁰ The result is that the parser prints the following (we repeat the user input on the first line):

```
> 2 3
2
syntax error
incorrect expression
0
```

This behavior gives a glimpse of the difficulties of good error recovery in Yacc. (See the exercises for more examples.) §

5.7.4 Error Recovery in TINY

The Yacc specification file **tiny.y** in Appendix B contains two error productions, one for **stmt** (line 4047) and one for **factor** (line 4139), whose associated actions are to return the null syntax tree. These error productions provide a level of error handling similar to that of the recursive-descent TINY parser in the previous chapter, except that no attempt is made to build a significant syntax tree in the presence of errors. Also, these error productions provide no special synchronization for the restart of the parse, so in the presence of multiple errors many tokens may be skipped.

EXERCISES

5.1 Consider the following grammar:

$$\begin{aligned} E &\rightarrow (L) \mid a \\ L &\rightarrow L , E \mid E \end{aligned}$$

- a. Construct the DFA of LR(0) items for this grammar.
 - b. Construct the SLR(1) parsing table.
 - c. Show the parsing stack and the actions of an SLR(1) parser for the input string $((a), a, (a, a))$.
 - d. Is this grammar an LR(0) grammar? If not, describe the LR(0) conflict. If so, construct the LR(0) parsing table, and describe how a parse might differ from an SLR(1) parse.
- 5.2 Consider the grammar of the previous exercise.
- a. Construct the DFA of LR(1) items for this grammar.
 - b. Construct the general LR(1) parsing table.
 - c. Construct the DFA of LALR(1) items for this grammar.
 - d. Construct the LALR(1) parsing table.

¹⁰ Some versions of Yacc pop the parsing stack again before deleting any input. This can result in even more complicated behavior. See the exercises.

- e. Describe any differences that might occur between the actions of a general LR(1) parser and an LALR(1) parser.

5.3 Consider the following grammar:

$$S \rightarrow S (S) \mid \varepsilon$$

- Construct the DFA of LR(0) items for this grammar.
 - Construct the SLR(1) parsing table.
 - Show the parsing stack and the actions of an SLR(1) parser for the input string $((()))$.
 - Is this grammar an LR(0) grammar? If not, describe the LR(0) conflict. If so, construct the LR(0) parsing table, and describe how a parse might differ from an SLR(1) parse.
- 5.4** Consider the grammar of the previous exercise.
- Construct the DFA of LR(1) items for this grammar.
 - Construct the general LR(1) parsing table.
 - Construct the DFA of LALR(1) items for this grammar.
 - Construct the LALR(1) parsing table.
 - Describe any differences that might occur between the actions of a general LR(1) parser and an LALR(1) parser.
- 5.5** Consider the following grammar of simplified statement sequences:

$$\begin{aligned} stmt\text{-}sequence &\rightarrow stmt\text{-}sequence ; stmt \mid stmt \\ stmt &\rightarrow s \end{aligned}$$

- Construct the DFA of LR(0) items for this grammar.
 - Construct the SLR(1) parsing table.
 - Show the parsing stack and the actions of an SLR(1) parser for the input string $s ; s ; s$.
 - Is this grammar an LR(0) grammar? If not, describe the LR(0) conflict. If so, construct the LR(0) parsing table, and describe how a parse might differ from an SLR(1) parse.
- 5.6** Consider the grammar of the previous exercise.
- Construct the DFA of LR(1) items for this grammar.
 - Construct the general LR(1) parsing table.
 - Construct the DFA of LALR(1) items for this grammar.
 - Construct the LALR(1) parsing table.
 - Describe any differences that might occur between the actions of a general LR(1) parser and an LALR(1) parser.
- 5.7** Consider the following grammar:

$$\begin{aligned} E &\rightarrow (L) \mid a \\ L &\rightarrow E L \mid E \end{aligned}$$

- Construct the DFA of LR(0) items for this grammar.
- Construct the SLR(1) parsing table.

- c. Show the parsing stack and the actions of an SLR(1) parser for the input string $((a)a(a\ a))$.
 - d. Construct the DFA of LALR(1) items by propagating the lookaheads through the DFA of LR(0) items.
 - e. Construct the LALR(1) parsing table.
- 5.8 Consider the following grammar

$$\begin{aligned} \text{declaration} &\rightarrow \text{type var-list} \\ \text{type} &\rightarrow \text{int} \mid \text{float} \\ \text{var-list} &\rightarrow \text{identifier, var-list} \mid \text{identifier} \end{aligned}$$

- a. Rewrite it in a form more suitable for bottom-up parsing.
 - b. Construct the DFA of LR(0) items for the rewritten grammar.
 - c. Construct the SLR(1) parsing table for the rewritten grammar.
 - d. Show the parsing stack and the actions of an SLR(1) parser for the input string $\text{int } x, y, z$ using the table of part (c).
 - e. Construct the DFA of LALR(1) items by propagating the lookaheads through the DFA of LR(0) items of part (b).
 - f. Construct the LALR(1) parsing table for the rewritten grammar.
- 5.9 Write down a formal description of the algorithm for constructing the DFA of LALR(1) items by propagating lookaheads through the DFA of LR(0) items. (This algorithm was informally described in Section 5.4.3.)
- 5.10 All the parsing stacks shown in this chapter have included both state numbers and grammar symbols (for clarity). However, the parsing stack need only store the state numbers—the tokens and nonterminals need not be stored in the stack. Describe the SLR(1) parsing algorithm if only the state numbers are kept in the stack.
- 5.11 a. Show that the following grammar is not LR(1):

$$A \rightarrow a A a \mid \varepsilon$$

- b. Is this grammar ambiguous? Why or why not?
- 5.12 Show that the following grammar is LR(1) but not LALR(1):

$$\begin{aligned} S &\rightarrow a A d \mid b B d \mid a B e \mid b A e \\ A &\rightarrow c \\ B &\rightarrow c \end{aligned}$$

- 5.13 Show that an LR(1) grammar that is not LALR(1) must have only reduce-reduce conflicts.
- 5.14 Show that a prefix of a right sentential form is a viable prefix if and only if it does not extend beyond the handle.
- 5.15 Can there be an SLR(1) grammar that is not LALR(1)? Why or why not?
- 5.16 Show that a general LR(1) parser will make no reduction before an error is declared, if the next input token cannot eventually be shifted.
- 5.17 Can an SLR(1) parser make more or fewer reductions than an LALR(1) parser before declaring error? Explain.

- 5.18** The following ambiguous grammar generates the same strings as the grammar of Exercise 5.3 (namely, all strings of nested parentheses):

$$A \rightarrow AA \mid (A) \mid \varepsilon$$

Will a Yacc-generated parser using this grammar recognize all legal strings? Why or why not?

- 5.19** Given a state in which there are two possible reductions (on different lookaheads), Yacc will choose one of the reductions as its default action. Describe the rule Yacc uses for making this choice. (Hint: Use the grammar of Example 5.16, page 222, as a test case.)
- 5.20** Suppose we remove the associativity and precedence specifications of the operators from the Yacc specification of Figure 5.14, page 238 (thus leaving an ambiguous grammar). Describe what associativity and precedences result from Yacc's default disambiguation rules.
- 5.21** As described in the footnote on page 250, some Yacc parsers pop the parsing stack again before discarding any input while in "error state." Describe the behavior of such a parser for the Yacc specification of Example 5.21, given the erroneous input `2 3`.
- 5.22** a. Trace the behavior of a panic mode error recovery mechanism as described in Section 5.7.2, using the parsing Table 5.11, page 235, and the string `(*2`.
b. Suggest an improvement to the behavior in part (a).
- 5.23** Suppose the rule for **command** in the Yacc calculator specification of Figure 5.10, page 228, is replaced by the following rule for a **list** starting nonterminal:

```
list    :    list '\n' {exit(0);}
        |    list exp '\n' {printf("%d\n", $2);}
        |    list error '\n' {yyerrok;}
        ;
```

and the **yylex** procedure in that figure has the line

```
if (c == '\n') return 0;
```

deleted.

- a. Explain the differences in the behavior of this version of the simple calculator from that of the version in Figure 5.10.
- b. Explain the reason for the **yyerrok** in the last line of this rule. Give an example to show what would happen if it were not there.
- 5.24** a. Suppose the rule for **command** in the Yacc calculator specification of Figure 5.10 is replaced by the following rule:

```
command : exp {printf("%d\n", $1);}
        | exp error {printf("%d\n", $1);}
        ;
```

Explain precisely the behavior of the Yacc parser, given the input `2 3`.

- b. Suppose the rule for **command** in the Yacc calculator specification of Figure 5.10 is replaced instead by the following rule:

```
command : exp {printf("%d\n", $1);}
        | error exp {printf("%d\n", $2);}
        ;
```

Explain precisely the behavior of the Yacc parser, given the input `2 3`.

- 5.25 Suppose the Yacc error production of Example 5.21, page 249, was replaced by the following rule:

```

factor : NUMBER          { $$ = $1; }
      | '(' exp ')'      { $$ = $2; }
      | error {yyerror; $$ = 0; }
      ;

```

- a. Explain the behavior of the Yacc parser given the erroneous input `2++3`.
 - b. Explain the behavior of the Yacc parser given the erroneous input `2 3`.
- 5.26 Compare the error recovery of the Yacc-generated TINY parser to that of the recursive-descent parser of the previous chapter, using the test programs of Section 4.5.3. Explain the differences in behavior.

PROGRAMMING EXERCISES

- 5.27 Rewrite the Yacc specification of Figure 5.10, page 235, to use the following grammar rules (instead of `scanf`) to calculate the value of a number (and, hence, do away with the `NUMBER` token):

```

number → number digit | digit
digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

```

- 5.28 Add the following to the Yacc integer calculator specification of Figure 5.10 (make sure they have the correct associativity and precedence):
- a. Integer division with the symbol `/`.
 - b. Integer mod with the symbol `%`.
 - c. Integer exponentiation with the symbol `^`. (Warning: This operator has higher precedence than multiplication and is right associative.)
 - d. Unary minus with the symbol `-`. (See Exercise 3.12.)
- 5.29 Redo the Yacc calculator specification of Figure 5.10 so that the calculator will accept floating-point numbers (and perform floating-point calculations).
- 5.30 Rewrite the Yacc calculator specification of Figure 5.10 so that it *distinguishes* between floating-point and integer values, rather than simply computing everything as integers or floating-point numbers. (Hint: A “value” is now a record with a flag indicating whether it is integer or floating point.)
- 5.31
- a. Rewrite Yacc calculator specification of Figure 5.10 so that it returns a syntax tree according to the declarations of Section 3.3.2.
 - b. Write a function that takes as a parameter the syntax tree produced by your code of part (a) and returns the calculated value by traversing the tree.
- 5.32 The simple error recovery technique suggested for the calculator program on page 248 was flawed in that it could cause stack overflow after many errors. Rewrite it to remove this problem.
- 5.33 Rewrite the Yacc calculator specification of Figure 5.10 to add the following useful error messages:

```

“missing right parenthesis,” generated by the string (2+3
“missing left parenthesis,” generated by the string 2+3)
“missing operator,” generated by the string 2 3
“missing operand,” generated by the string (2+

```

- 5.34 The following grammar represents simple arithmetic expressions in a LISP-like prefix notation:

$$\begin{aligned} \text{lexp} &\rightarrow \text{number} \mid (\text{op lexp-seq}) \\ \text{op} &\rightarrow + \mid - \mid * \\ \text{lexp-seq} &\rightarrow \text{lexp-seq lexp} \mid \text{lexp} \end{aligned}$$

For example, the expression $(* (-2) 3 4)$ has value -24 . Write a Yacc specification for a program that will compute and print the value of expressions in this syntax. (Hint: This will require rewriting the grammar, as well as the use of a mechanism for passing the operator to an *lexp-seq*.)

- 5.35 The following (ambiguous) grammar represents the simple regular expressions discussed in Chapter 2:

$$\begin{aligned} \text{rexp} &\rightarrow \text{rexp " | " rexp} \\ &\quad \mid \text{rexp rexp} \\ &\quad \mid \text{rexp "*" } \\ &\quad \mid \text{" (" rexp ")" } \\ &\quad \mid \text{letter} \end{aligned}$$

- a. Write a skeleton Yacc specification (i.e., without actions) that expresses the correct associativity and precedence of the operations.
 - b. Extend your specification of part (a) to include all actions and auxiliary procedures needed to produce a “regular expression compiler,” that is, a program that will take a regular expression as input and output a C program that, when compiled, will search an input string for the first occurrence of a substring that matches the regular expression. (Hint: A table, or two-dimensional array, can be used to represent the states and transitions of the associated NFA. The NFA can then be simulated using a queue to store states. Only the table needs to be generated by the Yacc actions; the rest of the code will be always the same. See Chapter 2.)
- 5.36 Rewrite the Yacc specification for TINY (Appendix B, lines 4000–4162) in a more compact form in either of the following ways:
- a. by using an ambiguous grammar for expressions (with Yacc disambiguating rules for precedence and associativity), or
 - b. by collapsing the recognition of operators into a single rule, as in

$$\begin{aligned} \text{exp} &\rightarrow \text{exp op term} \mid \dots \\ \text{op} &\rightarrow + \mid - \mid \dots \end{aligned}$$

and by using the **%union** Yacc declaration (allowing *op* to return the operator, while *exp* and other nonterminals return pointers to tree nodes).

Make sure your parser produces the same syntax trees as before.

- 5.37 Add the comparison operators $\leq$ (less than or equal to), $>$ (greater than), $\geq$ (greater than or equal to), and $\neq$ (not equal to) to the Yacc specification for the TINY parser. (This will require adding these tokens and changing the scanner as well, but should not require a change to the syntax tree.)
- 5.38 Add the Boolean operators **and**, **or**, and **not** to the Yacc specification for the TINY

parser. Give them the properties described in Exercise 3.5, as well as lower precedence than all arithmetic operators. Make sure any expression can be either Boolean or integer.

5.39 Rewrite the Yacc specification for the TINY parser to improve its error recovery.

NOTES AND REFERENCES

General LR parsing was invented by Knuth [1965], but it was thought to be impractical until the SLR and LALR techniques were developed by DeRemer [1969, 1971]. We have echoed the conventional wisdom that LR(1) parsers are too complicated for practical use. In fact it is possible to build practical LR(1) parsers using more delicate state merging techniques than those of LALR(1) parsing [Pager, 1977]. Nevertheless, the added power is rarely needed. A thorough study of the theory of LR parsing techniques can be found in Aho and Ullman [1972].

Yacc was developed by Steve Johnson at AT&T Bell Labs in the mid-1970s and is included in most Unix implementations [Johnson, 1975]. It was used to develop the portable C compiler [Johnson, 1978] and many other compilers. Bison was developed by Richard Stallman and others; Gnu Bison is a part of the Gnu software distribution of the Free Software Foundation and is available from many Internet sites. An example of the use of Yacc to develop a powerful, yet compact, calculator program is given in Kernighan and Pike [1984]. An in-depth study of the use of Yacc can be found in Schreiner and Friedman [1985].

LR error recovery techniques are studied in Graham, Haley, and Joy [1979]; Penello and DeRemer [1978]; and Burke and Fisher [1987]. An LR error repair technique is described in Fischer and LeBlanc [1991]. The basic ideas of the panic mode technique described in Section 5.7.2 are attributed by Fischer and LeBlanc to James [1972].